Inheritance

```
#include <iostream>
using namespace std;

class Person
{
protected:
    string name;

public:
    Person(string n)
    {
        name = n;
        cout << "Person Constructor" << endl;
    }

    ~Person()
    {
        cout << "Person Destructor" << endl;
    }
};

class Student : public Person
{
private:
    int rollNo;

public:
    Student(string n, int r) : Person(n)
    {
        rollNo = r;
        cout << "Student Constructor" << endl;
    }

    ~Student()
    {
        cout << "Student Destructor" << endl;
    }

    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
    }
};

int main()
{
    Student s("Rahul", 101);

    s.display();

    return 0;
}
```

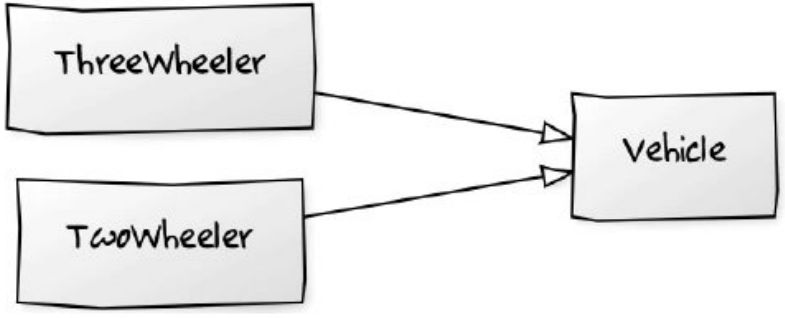
- A ConstructorB ConstructorC Constructor
- C DestructorB DestructorA Destructor

Easy rule for students:
Construction: Base → Derived
Destruction: Derived → Base

- We often find one object is a *specialization* / *generalization* of another
- OOAD models this using **ISA** relationship
- C++ models **ISA** relationship by *Inheritance* of classes

- Rose **ISA** Flower
  - Rose has the properties of Flower – like fragrance, having petals etc.
  - Rose has some additional properties – like rosy fragrance
  - Rose is a *specialization* of Flower
  - Flower is a *generalization* of Rose
- Red Rose **ISA** Rose
  - Red Rose has the properties of Rose – like rosy fragrance etc.
  - Red Rose has some additional properties – like it is red
  - Red Rose is a *specialization* of Rose
  - Rose is a *generalization* of Red Rose

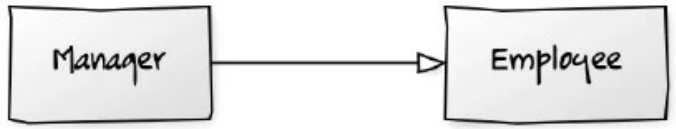




- TwoWheeler **ISA** Vehicle; ThreeWheeler **ISA** Vehicle
- Manager **ISA** Employee

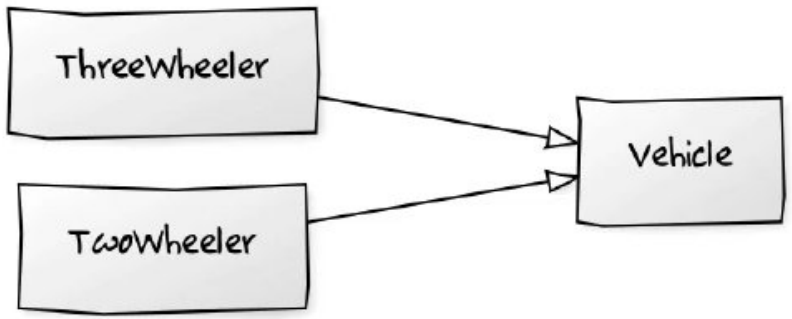


- Manager **ISA** Employee [**Single Inheritance**]




```
class Employee;                    // Base Class = Employee
class Manager: public Employee; // Derived Class = Manager; Base Class = Employee
```

- TwoWheeler **ISA** Vehicle; ThreeWheeler **ISA** Vehicle [**Hybrid Inheritance**]




```
class Vehicle;                        // Base Class = Vehicle -- Root
class TwoWheeler: public Vehicle;     // Derived Class = TwoWheeler; Base Class = Vehicle
class ThreeWheeler: public Vehicle;   // Derived Class = ThreeWheeler; Base Class = Vehicle
```

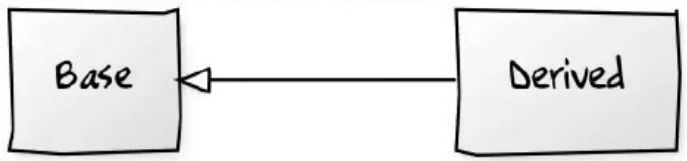
- Red Rose **ISA** Rose **ISA** Flower [**Multi-Level Inheritance**]




```
class Flower;                   // Base Class = Flower -- Root
class Rose: public Flower;    // Derived Class = Rose; Base Class = Flower
class RedRose: public Rose;   // Derived Class = RedRose; Base Class = Rose
```

- **Landline Phone**
  - Call: By dial / keyboard
  - Answer
  - Caller ID (with special attached device)
- **Mobile Phone**
  - Call: By keyboard – shows number
    - By Number
    - By Name
  - Answer
  - Caller ID
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name
- **Smart Phone**
  - Call: By touchscreen – shows number & photo
    - By Number
    - By Name
  - Answer
  - Caller ID
  - Redial
  - Set Ring Tone
  - Add Contact
    - Number
    - Name
    - Photo

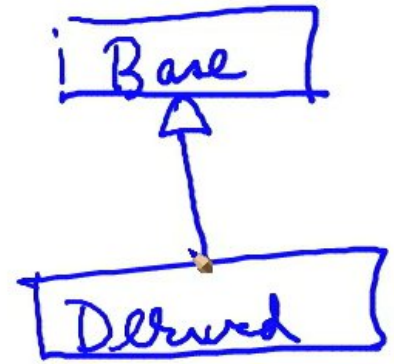
- Derived **ISA** Base




```
class Base;                          // Base Class = Base
class Derived: public Base; // Derived Class = Derived
```

- Use keyword `public` after class name to denote inheritance
- Name of the Base class follow the keyword

- Derived **ISA** Base ✓
- **Data Members**
  - Derived class *inherits* all data members of Base class
  - Derived class may *add* data members of its own



- **Member Functions**
  - Derived class *inherits* all member functions of Base class
  - Derived class may *override* a member function of Base class by *redefining* it with the same signature
  - Derived class may *overload* a member function of Base class by *redefining* it with the *same name*; but *different signature*
- **Access Specification**
  - Derived class *cannot access* private members of Base class
  - Derived class *can access* protected members of Base class
- **Construction-Destruction**
  - A *constructor* of the Derived class *must first* call a *constructor* of the Base class to construct the Base class instance of the Derived class
  - The *destructor* of the Derived class *must* call the *destructor* of the Base class to destruct the Base class instance of the Derived class

- Derived **ISA** Base
- **Data Members**
  - Derived class *inherits* all data members of Base class
  - Derived class may *add* data members of its own
- **Member Functions**
  - Derived class *inherits* all member functions of Base class
  - Derived class may *override* a member function of Base class by *redefining* it with the same signature
  - Derived class may *overload* a member function of Base class by *redefining* it with the *same name*; but *different signature*
- **Access Specification**
  - Derived class *cannot access* private members of Base class
  - Derived class *can access* protected members of Base class
- **Construction-Destruction**
  - A *constructor* of the Derived class *must first* call a *constructor* of the Base class to construct the Base class instance of the Derived class
  - The *destructor* of the Derived class *must* call the *destructor* of the Base class to destruct the Base class instance of the Derived class

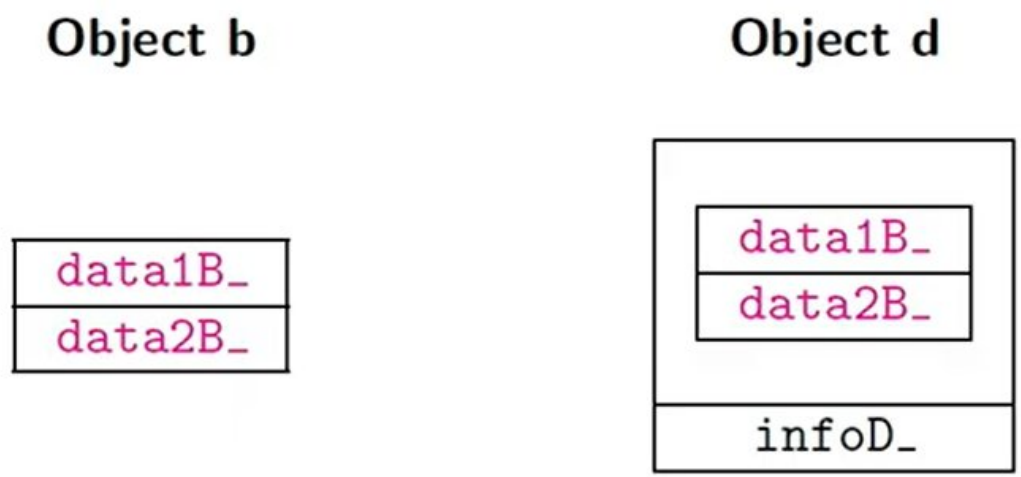
```
class B { // Base Class
    int data1B_;
public:
    int data2B_;
    // ...
};

class D: public B { // Derived Class
    // Inherits B::data1B_
    // Inherits B::data2B_
    int infoD_; // Adds D::infoD_
public:
    // ...
};

B b; // Base Class Object

D d; // Derived Class Object
```

Object Layout



- d *cannot access* data1B_ even though is a part of d!
- d *can access* data2B_

Inheritance

```
class B { public: // Base Class
    void f(int i);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)


    // Inherits B::g(int)

};
B b;
D d;

b.f(1); // Calls B::f(int)
b.g(2); // Calls B::g(int)

d.f(3); // Calls B::f(int)
d.g(4); // Calls B::g(int)
```

- D::f(int) overrides B::f(int)
- D::f(string&) overloads B::f(int)

Override & Overload

```
class B { public: // Base Class
    void f(int);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)
    void f(int);      // Overrides B::f(int)
    void f(string&); // Overloads B::f(int)
    // Inherits B::g(int)
    void h(int i);   // Adds D::h(int)
};
B b;
D d;

b.f(1);      // Calls B::f(int)
b.g(2);      // Calls B::g(int)

d.f(3);      // Calls D::f(int)
d.g(4);      // Calls B::g(int)

d.f("red"); // Calls D::f(string&)
d.h(5);      // Calls D::h(int)
```

- Derived **ISA** Base
- **Member Functions**
  - Derived class *inherits* all member functions of Base class
    - **Note**: Derived class *does not inherit* the Constructors and Destructor of Base class but *must have access to them*
  - Derived class may *override* a member function of Base class by *redefining* it with the same signature
  - Derived class may *overload* a member function of Base class by *redefining* it with the *same name*; but *different signature*
  - Derived class *may add* new member functions
- **Static Member Functions**
  - Derived class *does not inherit* the static member functions of Base class
- **Friend Functions**
  - Derived class *does not inherit* the friend functions of Base class

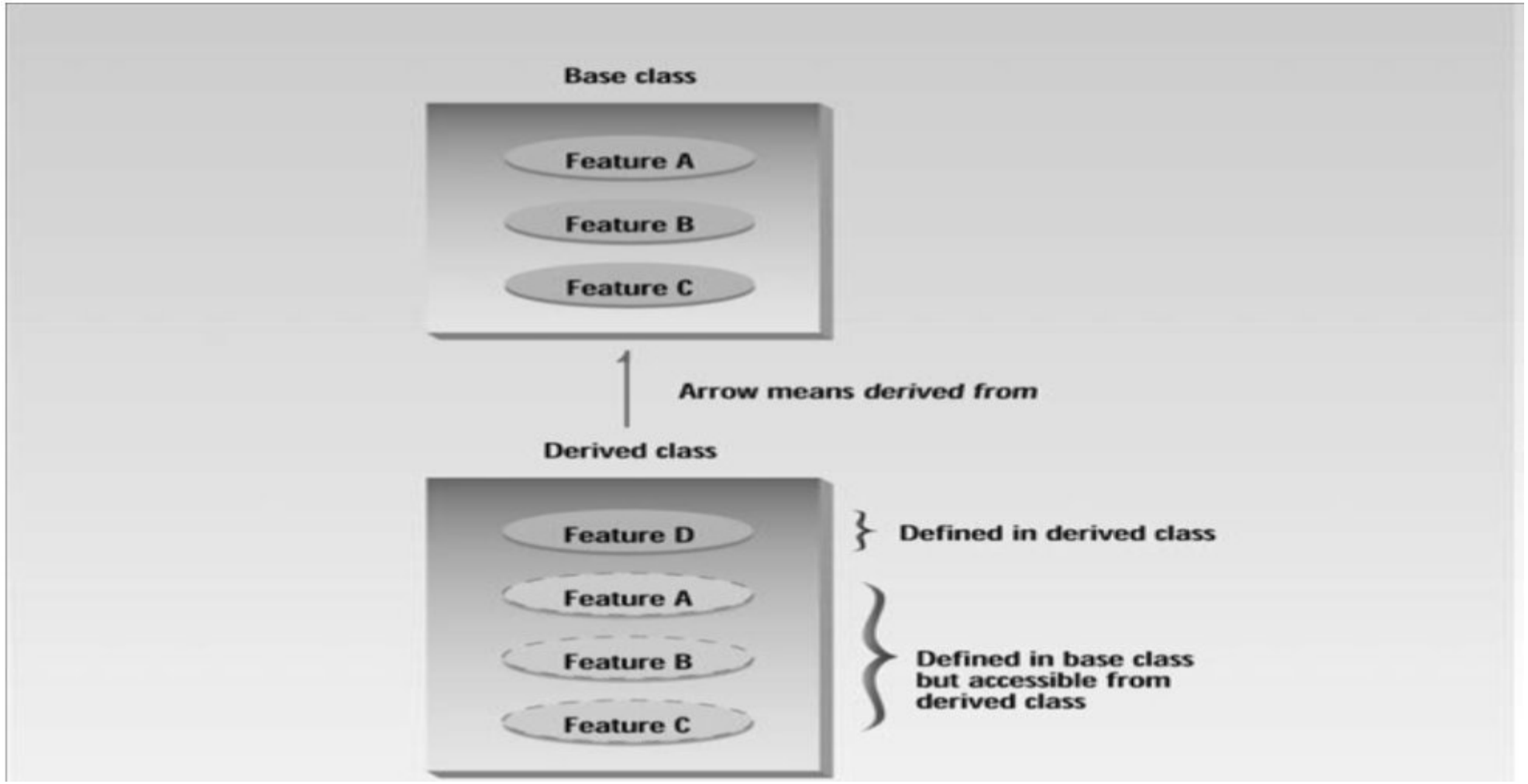
Basis	Function Overloading	Function Overriding
Name of Function	• All overloads have the same function name	• All overrides have the same function name
Signature	• Function signatures must be different	• Function signatures are same
Type of Function	• Can be global, friend, static or non-static member function	• Must be a non-static member function - non-virtual or virtual
Inheritance	• Can happen with or without inheritance	• Happens only with inheritance
Signature	• Function signatures must be different in number of parameters and / or their types	• Function signatures are same
Polymorphism	• Static (Compile time)	• Static (Compile time) or Dynamic (Runtime)
Scope	• Overloaded functions are in the same scope	• Functions are in different scopes (base clase and derived class)
Purpose	• To have multiple functions with same name that act differently depending on parameters	• To perform additional or different tasks than the base class function
Constructor	• Constructors can be overloaded	• Constructors cannot be overridden
Destructor	• The destructor cannot be overloaded	• The destructor cannot be overridden
Usage	• Can be overloaded multiple times	• Can be overridden once in the derived class

Introduction

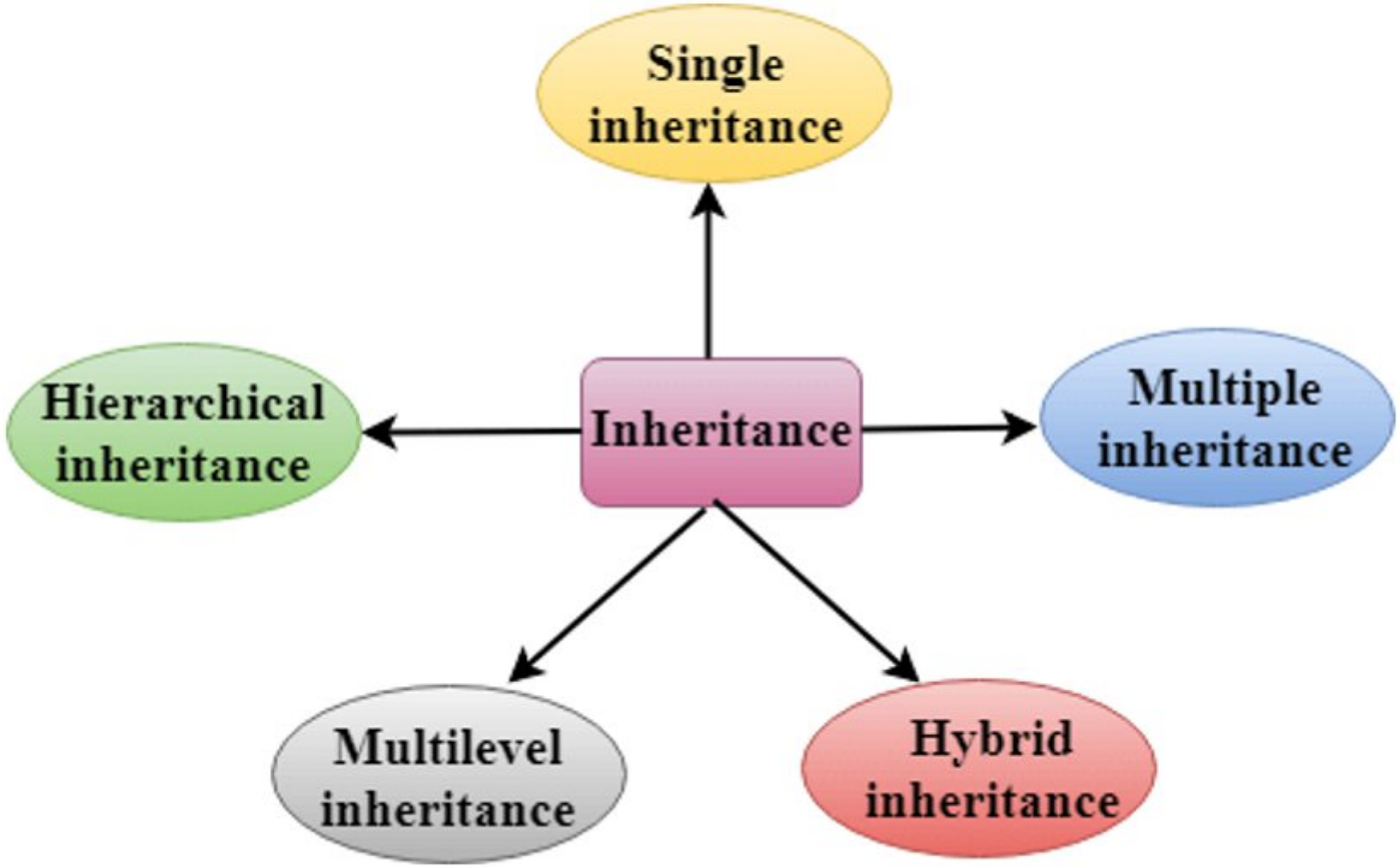
- **Reusability** is an important feature of OOPC. C++ strongly supports the concept of reusability.
- Reusing existing code **saves time** and **money** and increases a program's **reliability**
- The mechanism of deriving a new class from an old one is called **inheritance /derivation / extending classes**.
- The old class is referred to as **base class/superclass/parent class**. The new class is called the **derived class /subclass/child class**.

Introduction

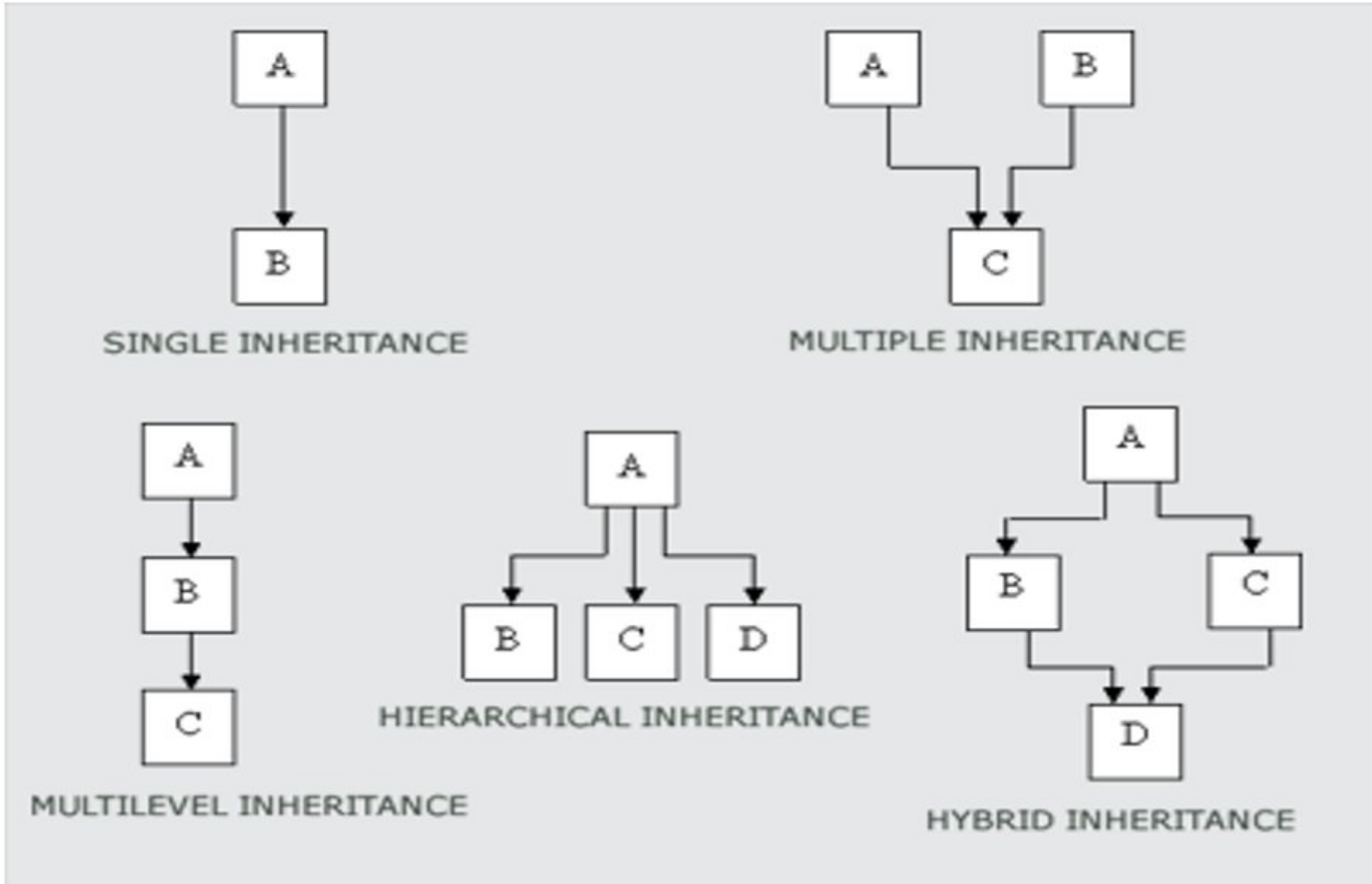
- The derived class inherits some or all of the traits from the base class.



Types of Inheritance



Types of Inheritance



Types of Inheritance

- **Single inheritance:** process of deriving a class from only **one base class**
- **Multilevel Inheritance:** process of deriving a class from **another derived class**
- **Multiple Inheritance:** process of deriving a class from **two or more base classes.**
- **Hierarchical Inheritance:** process of **deriving more than one class** from a one base class.
- **Hybrid Inheritance:** **combination** of more than one type of inheritance.

Defining Derived Classes

- A derived class can be defined by specifying its relationship with the base class in addition to its own details.

The colon indicates that the **derived-class-name** is derived from the **base-class-name**

```
class derived-class-name : visibility-mode base-class-name
{
    ……//
    ……//    members of derived class
    ……//
};
```

The visibility mode is optional and , if present, may be either **private** or **public**.

The default visibility mode is **private**.

Visibility mode specifies whether the features of the base class are **derived privately or publicly**.

Defining Derived Classes

```
//private derivation
class ABC: private XYZ
{
    members of ABC
};

//private derivation by default
class ABC: XYZ
{
    members of ABC
};

//public derivation
class ABC: public XYZ
{
    members of ABC
};
```

Making Private Member Inheritable

- **Private member** of a base class **cannot be accessible** in derived class directly.

- By modifying the visibility limit of the private member to **public** would make it **accessible to all the other functions** of the program, thus **taking away the advantage of data hiding.**

Making Private Member Inheritable

- C++ provides a third visibility modifier, **protected**, which is **accessible by the member functions within its class and any class immediately derived from it**.

- It can not be accessed by the functions outside these two classes.

Inheritance and Accessibility

```
class alpha
{
private:          //optional
    ........     //visible to member functions within its class
    ........
protected:
    ........     //visible to member functions of its own and derived class
    ........
public:
    ........     //visible to all functions in the program
    ........
};
```

Access Specifier	*Accessible from Own Class*	*Accessible from Derived Class*	*Accessible from Objects Outside Class*
public	yes	yes	yes
protected	yes	yes	no
private	yes	no	no

Making Private Member Inheritable

- The keywords private, protected and public may appear in any order and any number of times.

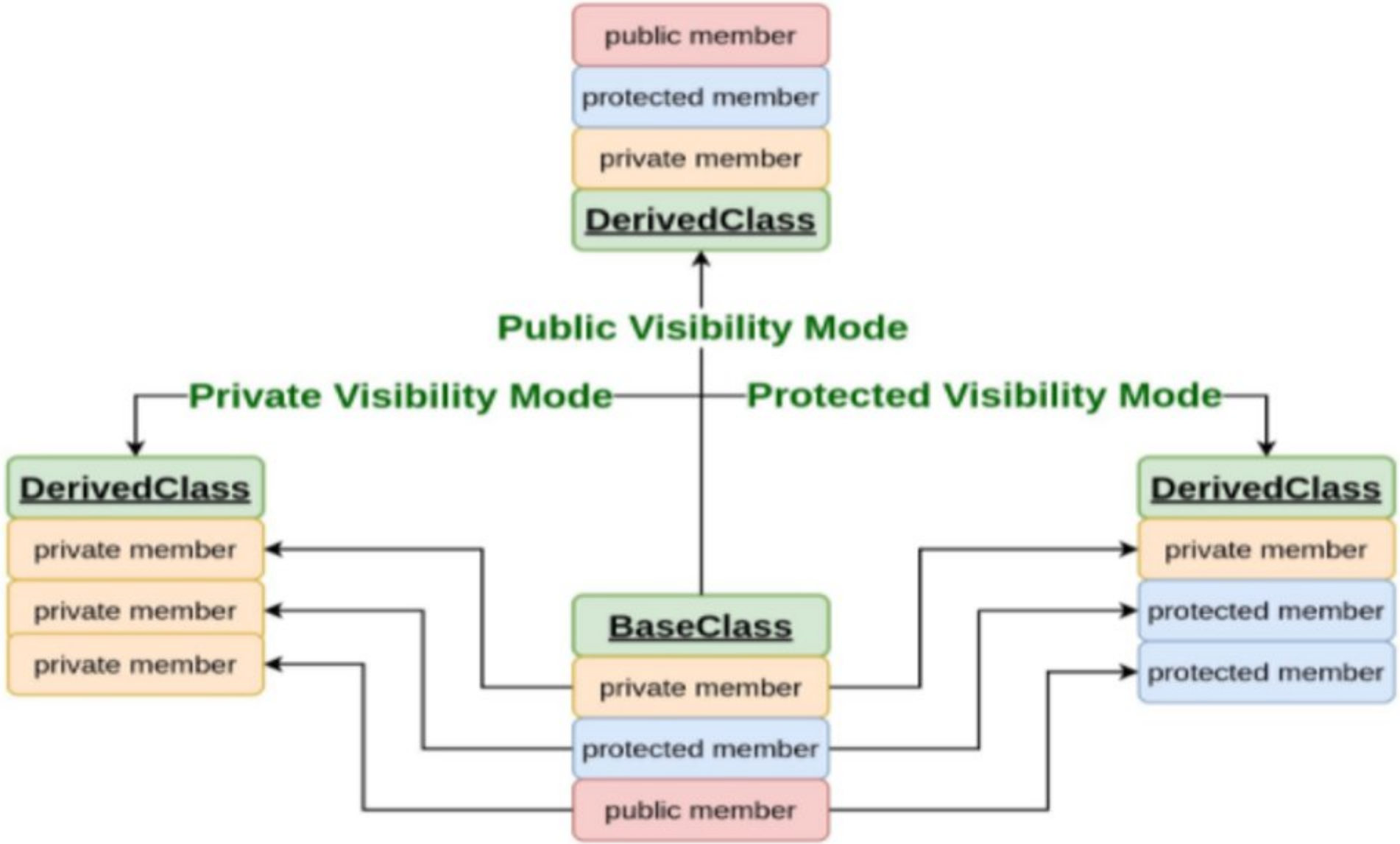
```
class alpha
{
protected:
     ........
public:
     ........
private:
     ........
public:
     ........
};
```

- However, the normal practice is to use them as follows:

```
class alpha
{
     ........ //private by default
     ........
protected:
     ........
public:
     ........
};
```

NOTE: It is also possible to inherit base class in protected mode (known as protected derivation)

Effect of inheritance on visibility of members



Visibility of inherited members

Base class member access specifier	Type of Inheritence		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Note: A derived class doesn't inherit access to private data members. However, it does inherit a full parent object, which contains any private members which that class declares.

private Access

```
class B {
private:  // Inaccessible to child
          // Inaccessible to others
    int data_;
public: // ...
    void Print() { cout << "B Object: ";
        cout << data_ << endl;
    }
};
class D: public B { int info_; public: // ...
    void Print() { cout << "D Object: ";
        cout << data_ << ", "; // Inaccessible
        cout << info_ <<endl;
    }
};
B b(0);
D d(1, 2);

b.data_ = 5;  // Inaccessible to all

b.Print();
d.Print();
```

- D::Print() *cannot access* B::data_ as it is private

protected Access

```
class B {
protected: // Accessible to child
          // Inaccessible to others
    int data_;
public: // ...
    void Print() { cout << "B Object: ";
        cout<<data_<<endl;
    }
};
class D: public B { int info_; public: // ...
    void Print() { cout << "D Object: ";
        cout << data_ << ", "; // Accessible
        cout << info_ << endl;
    }
};
B b(0);
D d(1, 2);

b.data_ = 5;  // Inaccessible to others

b.Print();
d.Print();
```

- D::Print() *can access* B::data_ as it is protected

```
class A                     //base class
{
//functions have the same access rules as the data shown here

private:                    //optional keyword
    int privdataA;          //visible to member functions within its class
protected:
    int protdataA;          //visible to member functions of its own and derived class
public:
    int pubdataA;           //visible to all functions in the program

    void funct()
    {
        int a;
        a = privdataA;      //OK
        a = protdataA;      //OK
        a = pubdataA;       //OK
    }
};
```

```
class B : public A  //publicly-derived class
{
public:
    void funct()
    {
        int a;
        //a = privdataA; //error: not accessible
        a = protdataA;   //OK
        a = pubdataA;    //OK
    }
};

class C : private A //privately-derived class
{
public:
    void funct()
    {
        int a;
        //a = privdataA; //error: not accessible
        a = protdataA;   //OK
        a = pubdataA;    //OK
    }
};
```

```
class D : protected A //protectedly-derived class
{
public:
    void funct()
    {
        int a;
        //a = privdataA; //error: not accessible
        a = protdataA;   //OK
        a = pubdataA;    //OK
    }
};
```

```
int main()
{
    int a;

    A objA;
    //a = objA.privdataA; //error: not accessible
    //a = objA.protdataA; //error: not accessible
    a = objA.pubdataA;    //OK

    B objB;
    //a = objB.privdataA; //error: not accessible
    //a = objB.protdataA; //error: not accessible
    a = objB.pubdataA;    //OK (A public to B)

    C objC;
    //a = objC.privdataA; //error: not accessible
    //a = objC.protdataA; //error: not accessible
    //a = objC.pubdataA;  //error: not accessible (A private to C)

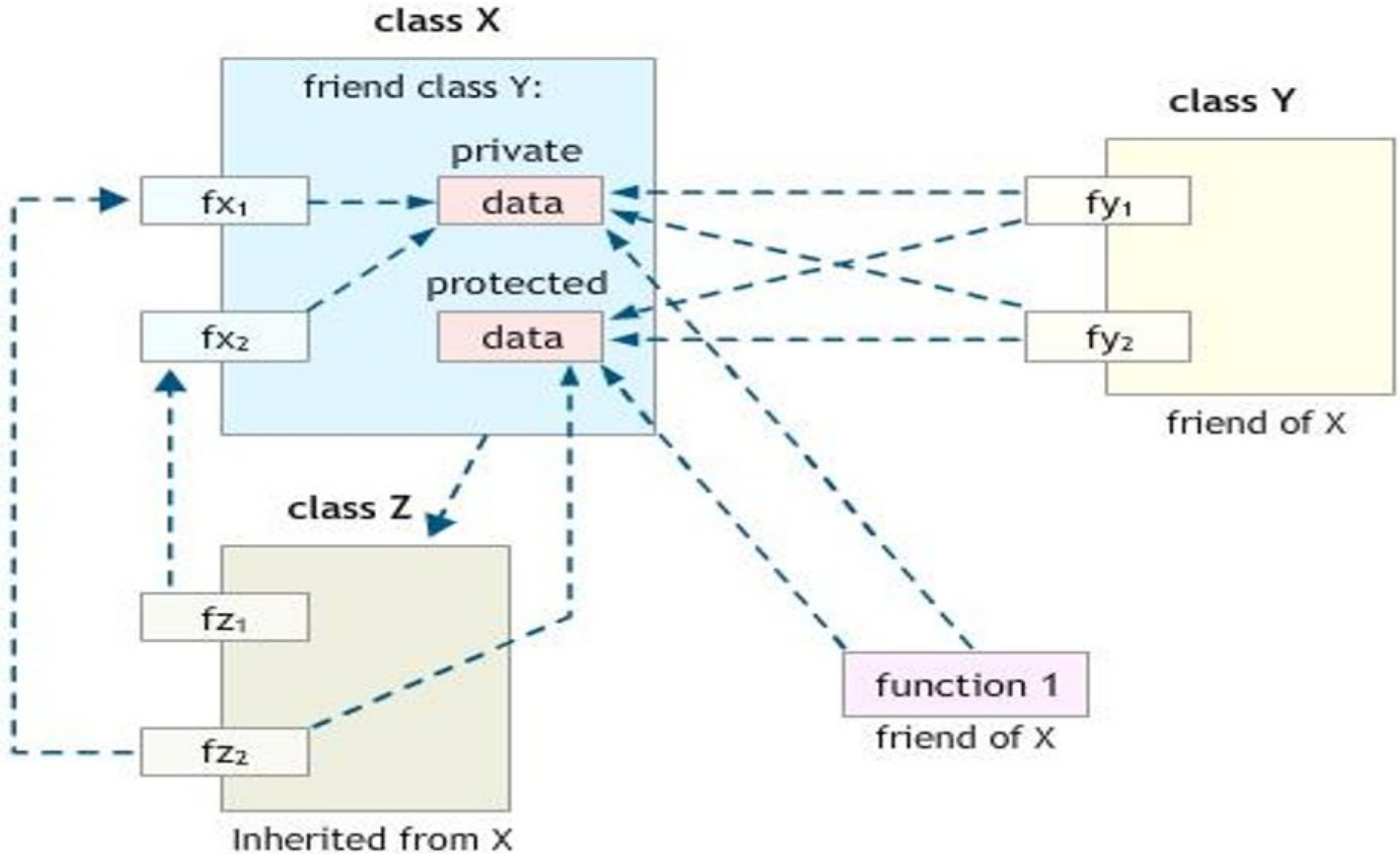
    D objD;
    //a = objD.privdataA; //error: not accessible
    //a = objD.protdataA; //error: not accessible
    //a = objD.pubdataA;  //error: not accessible (A protected to D)
}
```

Access Control to Data Members

Functions that can have access to the private and protected members of a class:

- A function that is a friend of the class.
- A member function of a class that is a friend of the class.
- A member function of a derived class.

Access Control to Data Members



Single inheritance: public derivation

```
#include<iostream>
using namespace std;

class B
{
    int a;
public:
    int b;
    void get_ab();
    int get_a();
    void show_a();
};

class D:public B
{
    int c;
public:
    void mul();
    void display();
};

void B::get_ab()
{
    a=5;   b=10;
}

int B::get_a()
{
    return a;
}

void B::show_a()
{
    cout<<"a="<<a<<endl<<endl;
}

void D::mul()
{
    c=b*get_a();
}

void D::display()
{
    cout<<"a="<<get_a()<<endl;
    cout<<"b="<<b<<endl;
    cout<<"c="<<c<<endl<<endl;
}
```

Single inheritance: public derivation

```
int main()
{
    D d;

    d.get_ab();
    d.mul();
    d.show_a();
    d.display();

    d.b=20;
    d.mul();
    d.display();
}
```

```
a=5

a=5
b=10
c=50

a=5
b=20
c=100
```

Single inheritance: private derivation

```
#include<iostream>
using namespace std;

class B
{
    int a;
public:
    int b;
    void get_ab();
    int get_a();
    void show_a();
};

class D:private B
{
    int c;
public:
    void mul();
    void display();
};

void B::get_ab()
{
    cout<<"Enter value for a and b:";
    cin>>a>>b;
}
int B::get_a()
{
    return a;
}
void B::show_a()
{
    cout<<"a="<<a<<endl;
}

void D::mul()
{
    get_ab();
    c=b*get_a();
}
void D::display()
{
    show_a();
    cout<<"b="<<b<<endl;
    cout<<"c="<<c<<endl<<endl;
}
```

Single inheritance: private derivation

```
int main()
{
    D d;

    //d.get_ab();   ERROR
    d.mul();
    //d.show_a();   ERROR
    d.display();

    //d.b=20;       ERROR
    d.mul();
    d.display();
}
```

```
Enter value for a and b:5    10
a=5
b=10
c=50

Enter value for a and b:12   20
a=12
b=20
c=240
```

Single inheritance: private derivation

```
int main()
{
    D d;

    //d.get_ab();   ERROR
    d.mul();
    //d.show_a();   ERROR
    d.display();

    //d.b=20;       ERROR
    d.mul();
    d.display();
}
```

```
Enter value for a and b:5    10
a=5
b=10
c=50

Enter value for a and b:12   20
a=12
b=20
c=240
```

Practical :Single Inheritance

Define a **Base Class Vegetable** having data member **Color** and member function **getdata()** which takes color as an input and **putdata()** which print the color as an output. Vegetable Class has one **subclass named Tomato** having data members **weight** and **size** and member function **gtdata()** which takes weight and size as an input and **ptdata()** which prints weight and size as output. Write a C++ Program which inherits the data of Vegetable class in Tomato class using **Single Inheritance**.

Practical :Single Inheritance

```
#include<iostream>
using namespace std;

class vegetable
{
    char color[10];
public:
    void getdata()
    {
        cout<< "ENter the color of vegetable : ";
        cin>> color;
    }
    void putdata()
    {
        cout<< "Vegetable is of " << color << " color." <<endl;
    }
};
```

Practical :Single Inheritance

```
class tomato : public vegetable
{
    float weight;
    char size[5];
public:
    void gtdata()
    {
        cout<< "Enter the weight of tomato : ";
        cin>> weight;
        cout<< "Enter the size of tomato (small/medium/big) : ";
        cin>> size;
    }
    void ptdata()
    {
        cout<<"It is tomato with weight "<<weight<<" and "<<size<<" in size";
    }
};

int main()
{
    tomato t;
    t.getdata();
    t.gtdata();
    t.putdata();
    t.ptdata();
}
```

```
ENter the color of vegetable : Red
Enter the weight of tomato : 10
Enter the size of tomato (small/medium/big) : small
Vegetable is of Red color.
It is tomato with weight 10 and small in size
```

Multilevel inheritance

- The **class A** serves as a base class for the derived **class B**, which in turn serves as a base class for the derived **class C.**
- The class B is known as **intermediate base class** since it provides a link for the inheritance between A and C.
- The chain ABC is known as **inheritance path.**

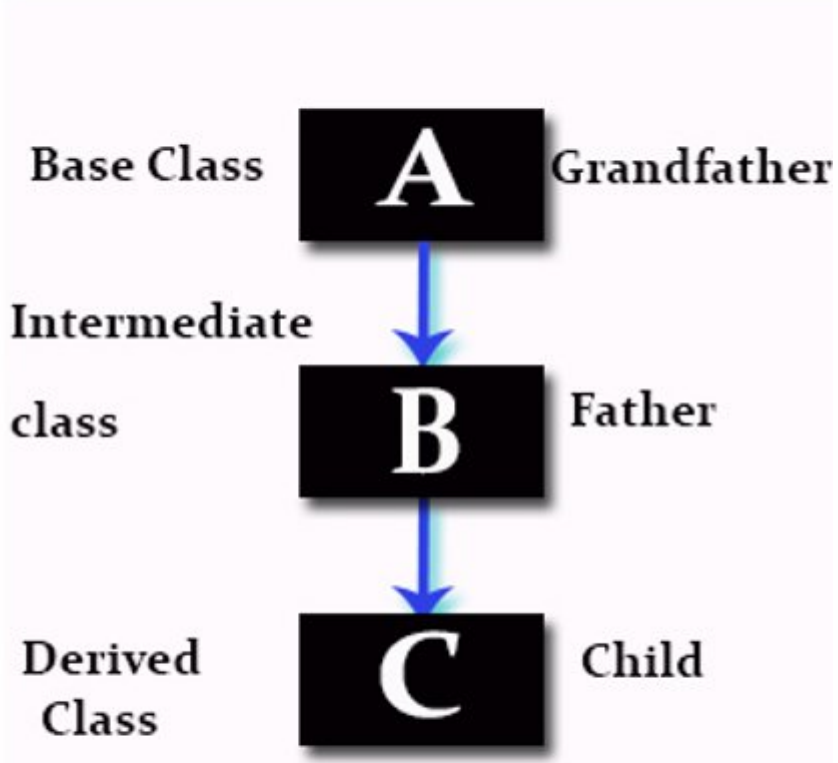


Fig: Multilevel Inheritance

NOTE: The process can be extended to any number of levels

Multilevel inheritance

```
class student
{
    int roll_number;
public:
    void get_number(int);
    void put_number(void);
};

void student :: get_number(int a)
{
    roll_number = a;
}

void student :: put_number()
{
    cout << "Roll Number: " << roll_number << endl;
}
```

```
class test : public student //First level derivation
{
protected:
    float sub1;
    float sub2;
public:
    void get_marks(float, float);
    void put_marks(void);
};

void test :: get_marks(float x, float y)
{
    sub1 = x;
    sub2 = y;
}

void test :: put_marks()
{
    cout << "Marks in Sub1 = " << sub1 << endl;
    cout << "Marks in Sub2 = " << sub2 << endl;
}
```

Multilevel inheritance

```
class result : public test    //second Level Derivation
{
    float total;
public:
    void display();
};
void result :: display(void)
{
    total = sub1 + sub2;
    cout << "Total = " << total << endl;
}

int main()
{
    result student1;

    student1.get_number(222);
    student1.get_marks(71.5, 89.5);

    student1.put_number();
    student1.put_marks();
    student1.display();
}
```

```
Roll Number: 222
Marks in Sub1 = 71.5
Marks in Sub2 = 89.5
Total = 161
```

Create a **class shape** having data member **shape_name** and member function to **get** and **print** shape_name. Derive a **Class Circle** which is inherited publicly from class shape and having data members **radius** of a circle and member function to **get** and **print** radius of a circle. Derive a **Class Area** which is inherited publicly from Class Circle and having data members **area_of_circle** and member function **display** () which displays area of a circle. Use object of class Area in main () function and get and display all the information.

Use the concepts of **Multilevel Inheritance.**

Practical : Multilevel Inheritance

```
#include<iostream>
using namespace std;

class shape
{
    char shape_name[10];
public:
    void getdata()
    {
        cout<< "Enter the shape name : ";
        cin>>shape_name;
    }
    void putdata()
    {
        cout<< "Your shape is " <<shape_name<< ".";
    }
};
```

Practical : Multilevel Inheritance

```
class circle : public shape
{
protected:
    int radius;
public:
    void gcdata()
    {
        cout<< "Enter the radius of circle : ";
        cin>>radius;
    }
    void pcdata()
    {
        cout<< "Circle is having " <<radius<< " radius.";
    }
};
```

```
class area : public circle
{
    float area_of_circle;
public:
    void display()
    {
        area_of_circle=3.14*radius*radius;
        cout<< "Area of circle is " <<area_of_circle<< ".";
    }
};
```

```
int main()
{
    area a;
    a.getdata();
    a.gcdata();
    a.putdata();
    a.pcdata();
    a.display();
    return 0;
}
```

```
Enter the shape name : circle
Enter the radius of circle : 5
Your shape is circle.Circle is having 5 radius.Area of circle is 78.5.
```

Multiple Inheritance

- A class can inherit the attributes of two or more classes is known as **multiple inheritance**.
- It allows us to combine the features of several existing classes as a starting point for defining new classes.
- It is like a **child** inheriting the **physical features** of one parent and the **intelligence** of another.

```
class D:visibility B1, visibility B2
{
    //Body of D
};
```

Multiple Inheritance

```
class M
{
protected:
    int m;
public:
    void get_m(int);
};

class N
{
protected:
    int n;
public:
    void get_n(int);
};

class P : public M, public N
{
public:
    void display(void);
};
```

```
void M :: get_m(int x)
{
    m=x;
}

void N :: get_n(int y)
{
    n=y;
}

void P :: display()
{
    cout<<"m= "<<m<<endl;
    cout<<"n= "<<n<<endl;
    cout<<"m*n= "<<m*n<<endl;
}

int main()
{
    P p;
    p.get_m(10);
    p.get_n(20);
    p.display();
}
```

```
m=10
n=20
m*n=200
```

Ambiguity Resolution in Inheritance

- **Ambiguity** may arise in inheritance application, when the **function in derived class overrides the inherited function.**
- A call to function with same name in base class and derived class by **derived class object** will **invoke function defined in derived class only**.
- However, we may invoke the function defined in base class by using the **scope resolution operator**.

Ambiguity Resolution in Inheritance

In Single Inheritance

```
class A
{
public:
     void display(void)
     {
          cout << "Class A \n";
     }
};

class B : public A
{
public:
     void display(void)
     {
          cout << "Class B \n";
     }
};

int main( )
{
    B b;                  // derived class object
    b.display( );         // invokes display() in B
    b.A::display( );      // invokes display() in A
    b.B::display( );      // invokes display() in B
}
```

```
Class B
Class A
Class B
```

Ambiguity Resolution in Inheritance

- We may also face a problem in using the **multiple inheritance**, when a **function with the same name** appears in more than one base class.
- We can solve this problem by defining a named instance within derived class, using the **class resolution operator** with the function.

Ambiguity Resolution in Inheritance

In Multiple Inheritance

```
class M
{
public:
     void display (void)
     {
          cout << "Class M \n";
     }
};

class N
{
public:
     void display (void)
     {
          cout << "Class N \n";
     }
};
```

```
class P : public M, public N
{
public:
     void display (void)
     {
          M :: display();
     }
};

int main( )
{
     P p;
     p.display( );
}
```

```
Class M
```

Interface Concept in C++

C++ does **not have a separate `interface` keyword** like Java. Instead, an interface is usually created using an **abstract class** containing **pure virtual functions**.

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    virtual void start() = 0;   // Pure virtual function
    virtual void stop() = 0;
};
```

```
class Car : public Vehicle {
public:
    void start() override {
        cout << "Car starts" << endl;
    }

    void stop() override {
        cout << "Car stops" << endl;
    }
};

int main() {
    Car c;
    c.start();
    c.stop();
}
```

Interface = contract that specifies what a class must do.

In C++, it is commonly implemented using an abstract class.

Functions are declared as pure virtual functions using = 0.

A class containing a pure virtual function cannot be instantiated.

The derived class must override the pure virtual functions.

Same Example in Java using Interface

```
interface Vehicle {
    void start();
    void stop();
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }

    public void stop() {
        System.out.println("Car stops");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car c = new Car();

        c.start();
        c.stop();
    }
}
```

C++

Uses **abstract class** to create interface-like behavior

Pure virtual function: virtual void start() = 0;

Inheritance: class Car : public Vehicle

Override using override (recommended)

Java

Uses **interface** keyword

Method: void start();

Implementation: class Car implements Vehicle

Override using @Override (recommended)

Real-time use cases of Interface in C++

1.Payment Gateway

1. Interface: Payment
2. Implementations: CreditCard, UPI, PayPal
3. Common function: pay()

2.Vehicle Control System

1. Interface: Vehicle
2. Implementations: Car, Bike, Bus
3. Common functions: start(), stop(), accelerate()

3.Notification System

1. Interface: Notification
2. Implementations: Email, SMS, WhatsApp
3. Common function: send()

4.File Storage

1. Interface: Storage
2. Implementations: LocalStorage, CloudStorage, DatabaseStorage
3. Common functions: save(), load()

5.AI/ML Model

1. Interface: Model
2. Implementations: CNNModel, SVMModel, DecisionTree
3. Common functions: train(), predict()

```
#include <iostream>
using namespace std;

class Payment {
public:
   virtual void pay(double amount) = 0;  // Interface
};

class UPI : public Payment {
public:
   void pay(double amount) override {
      cout << "Paid Rs." << amount << " using UPI" <<
endl;
   }
};
```

```
class CreditCard : public Payment {
public:
   void pay(double amount) override {
      cout << "Paid Rs." << amount << " using Credit Card" << endl;
   }
};

int main() {
   Payment *p;

   UPI u;
   p = &u;
   p->pay(500);

   CreditCard c;
   p = &c;
   p->pay(1000);
}
```

Concept: The application only knows **`Payment` → `pay()`**. It doesn't need to know *how* UPI or Credit Card performs the payment.

Interface = Common contract + Different implementations

This is particularly useful for explaining **abstraction + runtime polymorphism** together.

A **helper class** is a class created to perform **supporting or utility tasks** for another class or a part of a program.

```
#include <iostream>
using namespace std;

// Helper class
class DiscountHelper {
public:
    static double calculateDiscount(double amount) {
        if (amount >= 5000)
            return amount * 0.20;   // 20% discount
        else if (amount >= 2000)
            return amount * 0.10;   // 10% discount
        else
            return 0;
    }
};
```

```
// Main class
class Order {
    double amount;
public:
    Order(double a) {
        amount = a;
    }

    void generateBill() {
        double discount =
DiscountHelper::calculateDiscount(amount);

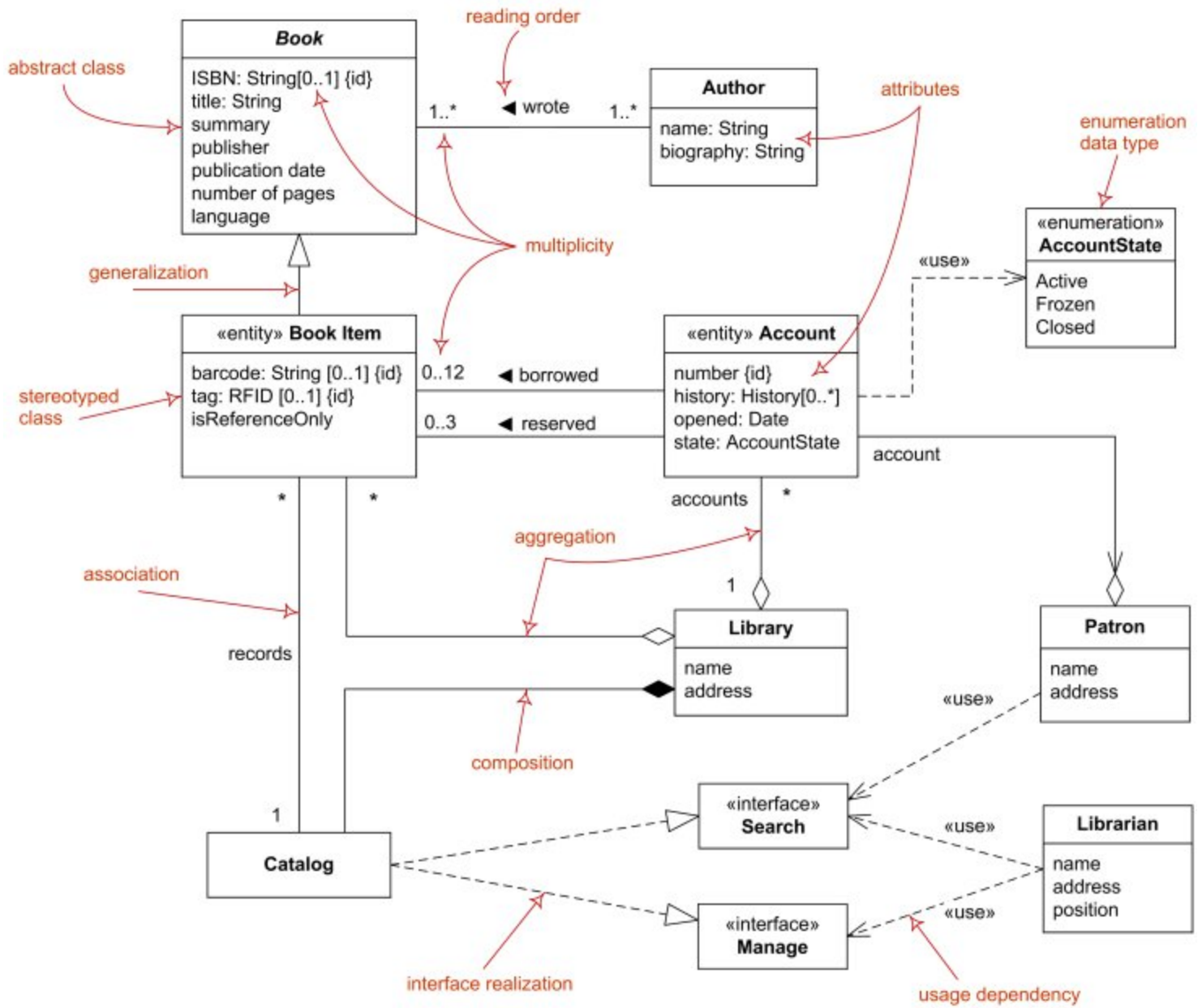
        cout << "Order Amount : Rs." << amount << endl;
        cout << "Discount     : Rs." << discount << endl;
        cout << "Final Amount : Rs." << amount - discount << endl
    }
};
int main() {
    Order order(6000);
    order.generateBill();

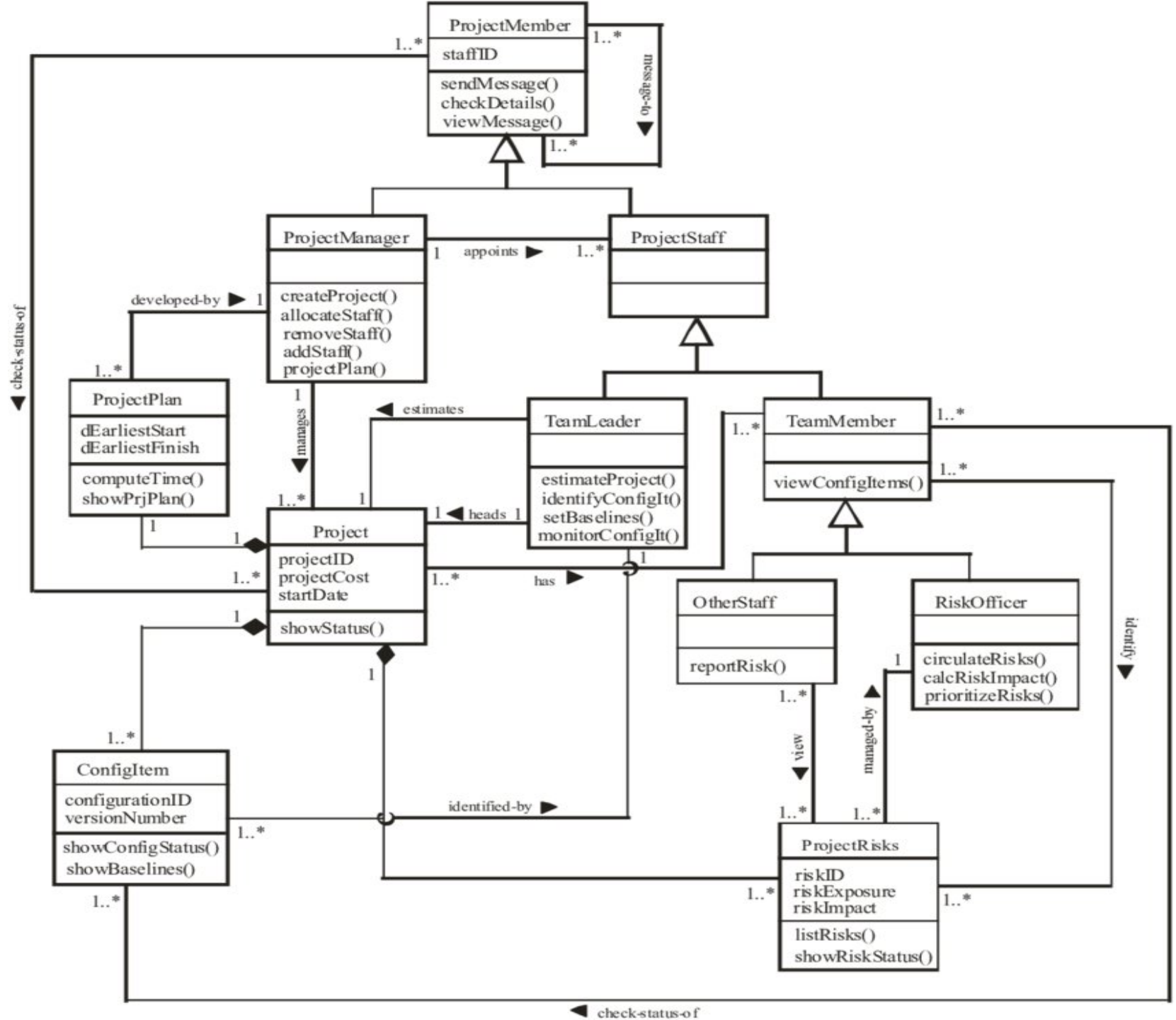
    return 0;
}
```

```
class B {
public:
    B()  { cout << "B "; }
    ~B() { cout << "~B "; }
};
class C {
public:
    C()  { cout << "C "; }
    ~C() { cout << "~C "; }
};
class D : public B {
    C data_; // Embedded Object
public:
    D()  { cout << "D " << endl; } // Intrinsic Base Class Object
    ~D() { cout << "~D "; }
};
int main() {
    D d;
}
```

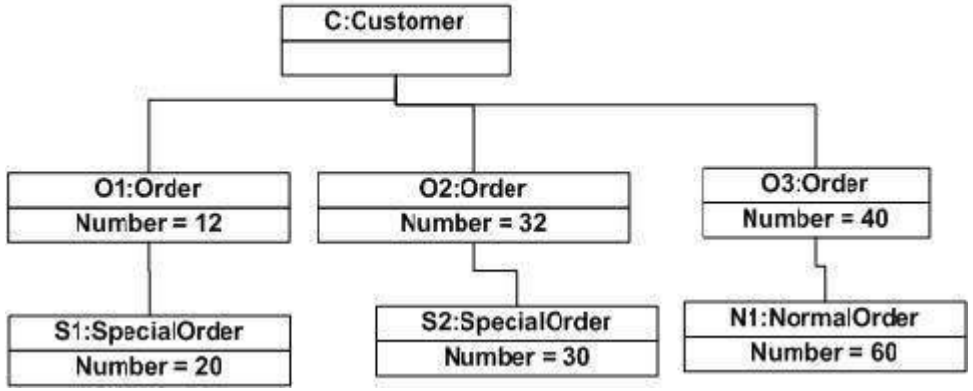
Output:

```
B C D       // First base class object, then embedded object, finally self
~D ~C ~B
```

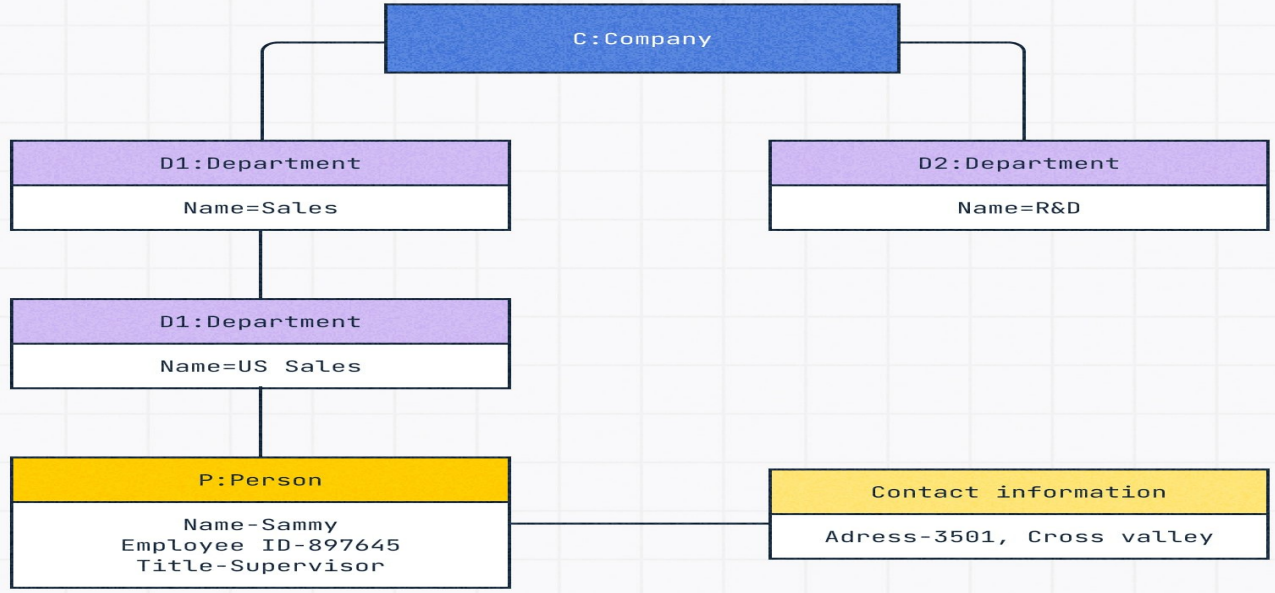

Book
ISBN: String[0..1] {id}
title: String
summary
publisher
publication date
number of pages
language
abstract class
reading order
1..*
wrote
1..*
Author
name: String
biography: String
attributes
enumeration data type
«enumeration» AccountState
Active
Frozen
Closed
«use»
multiplicity
generalization
«entity» Book Item
barcode: String [0..1] {id}
tag: RFID [0..1] {id}
isReferenceOnly
stereotyped class
0..12
borrowed
0..3
reserved
«entity» Account
number {id}
history: History[0..*]
opened: Date
state: AccountState
account
accounts
*
*
*
aggregation
association
1
Library
name
address
Patron
name
address
records
composition
«use»
1
«interface» Search
«use»
Librarian
name
address
position
Catalog
«interface» Manage
«use»
interface realization
usage dependency

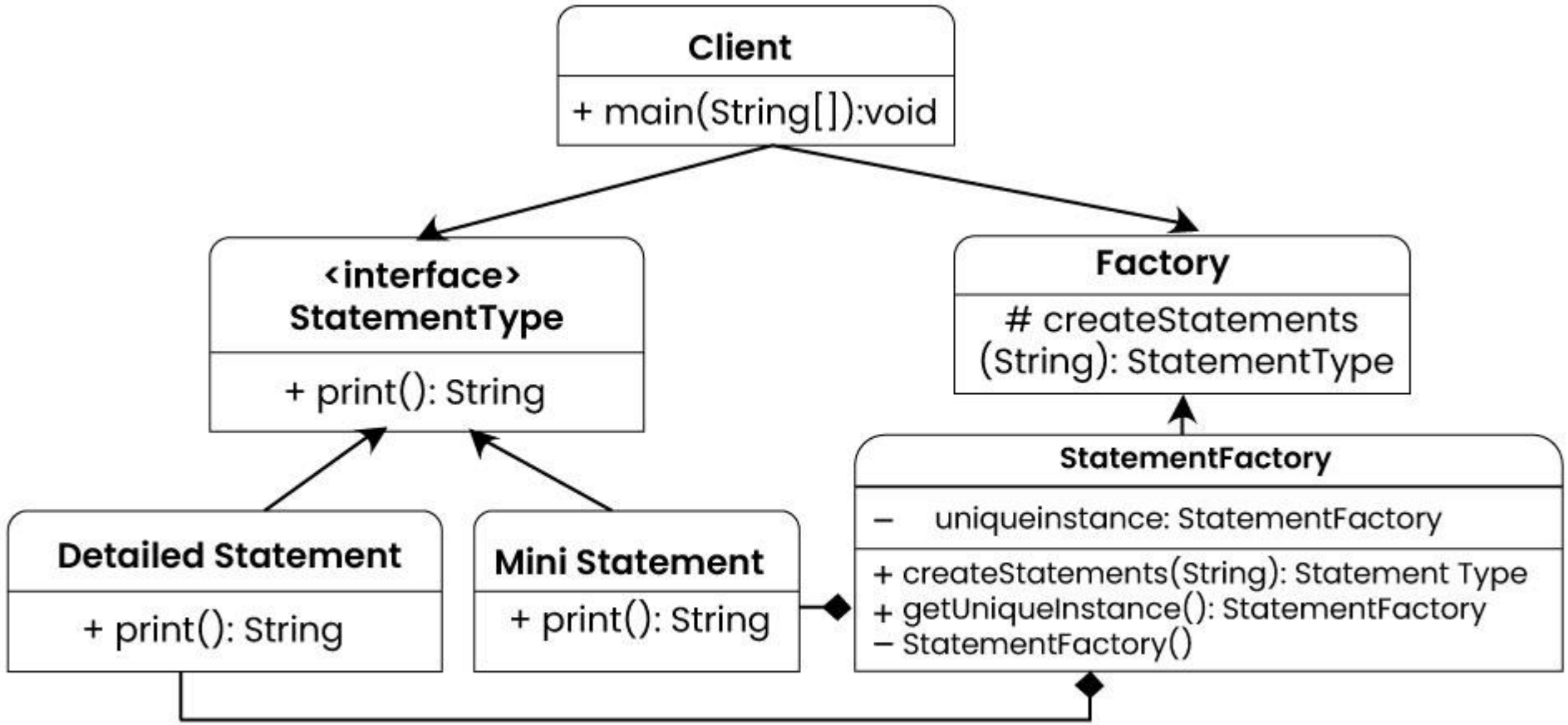

ProjectMember
staffID
sendMessage()
checkDetails()
viewMessage()
message-to
ProjectManager
createProject()
allocateStaff()
removeStaff()
addStaff()
projectPlan()
appoints
ProjectStaff
developed-by
check-status-of
ProjectPlan
dEarliestStart
dEarliestFinish
computeTime()
showPrjPlan()
manages
estimates
TeamLeader
estimateProject()
identifyConfigIt()
setBaselines()
monitorConfigIt()
heads
TeamMember
viewConfigItems()
Project
projectID
projectCost
startDate
showStatus()
has
OtherStaff
reportRisk()
RiskOfficer
circulateRisks()
calcRiskImpact()
prioritizeRisks()
identify
view
managed-by
ConfigItem
configurationID
versionNumber
showConfigStatus()
showBaselines()
identified-by
ProjectRisks
riskID
riskExposure
riskImpact
listRisks()
showRiskStatus()
check-status-of

Object diagram of an order management system



Object diagram of a company structure





Key Component of Singleton Method Design Pattern in JavaScript

Design Pattern



Reusable solutions to common software design problems



Improve code structure, flexibility, and maintainability



Provide proven templates for object creation, interaction, & architecture

Types of Design Pattern



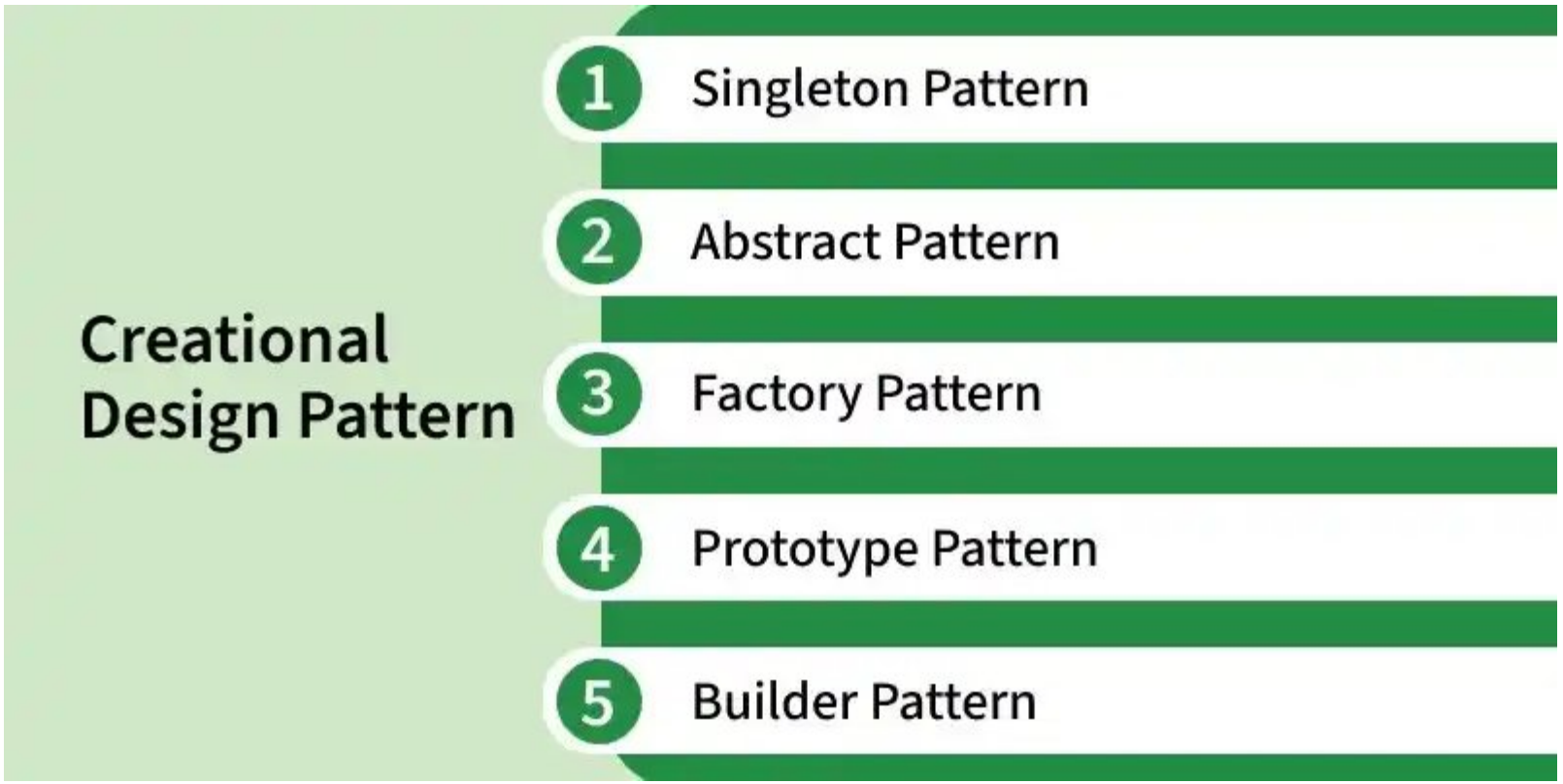
Creational Design Patterns



Structural Design Patterns



Behavioral Design Patterns


Creational
Design Pattern
1
Singleton Pattern
2
Abstract Pattern
3
Factory Pattern
4
Prototype Pattern
5
Builder Pattern

Structural Design Patterns

01 Adapter Pattern

02 Bridge Pattern

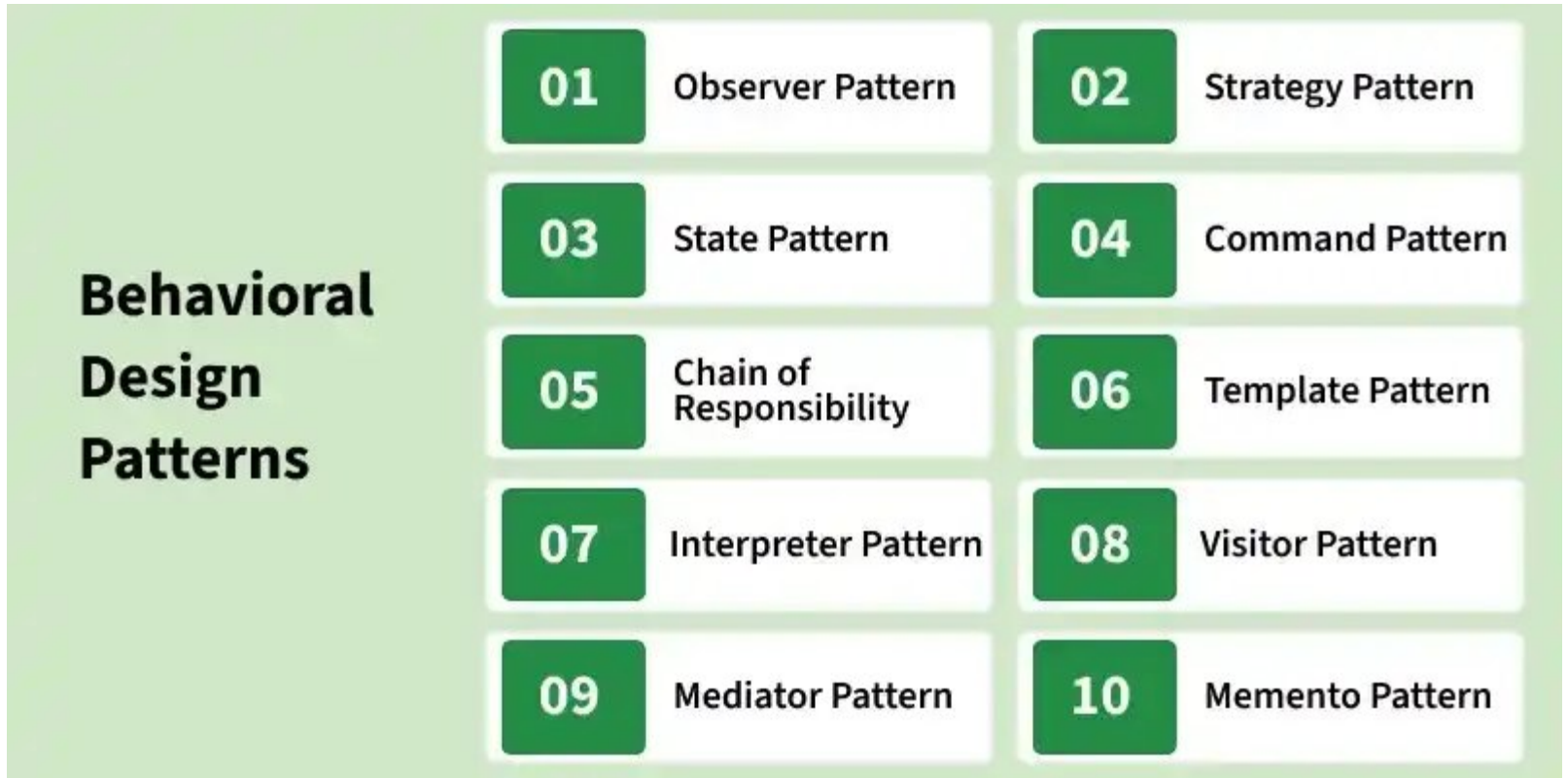
03 Composite Pattern

04 decorator Pattern

05 Facade Pattern

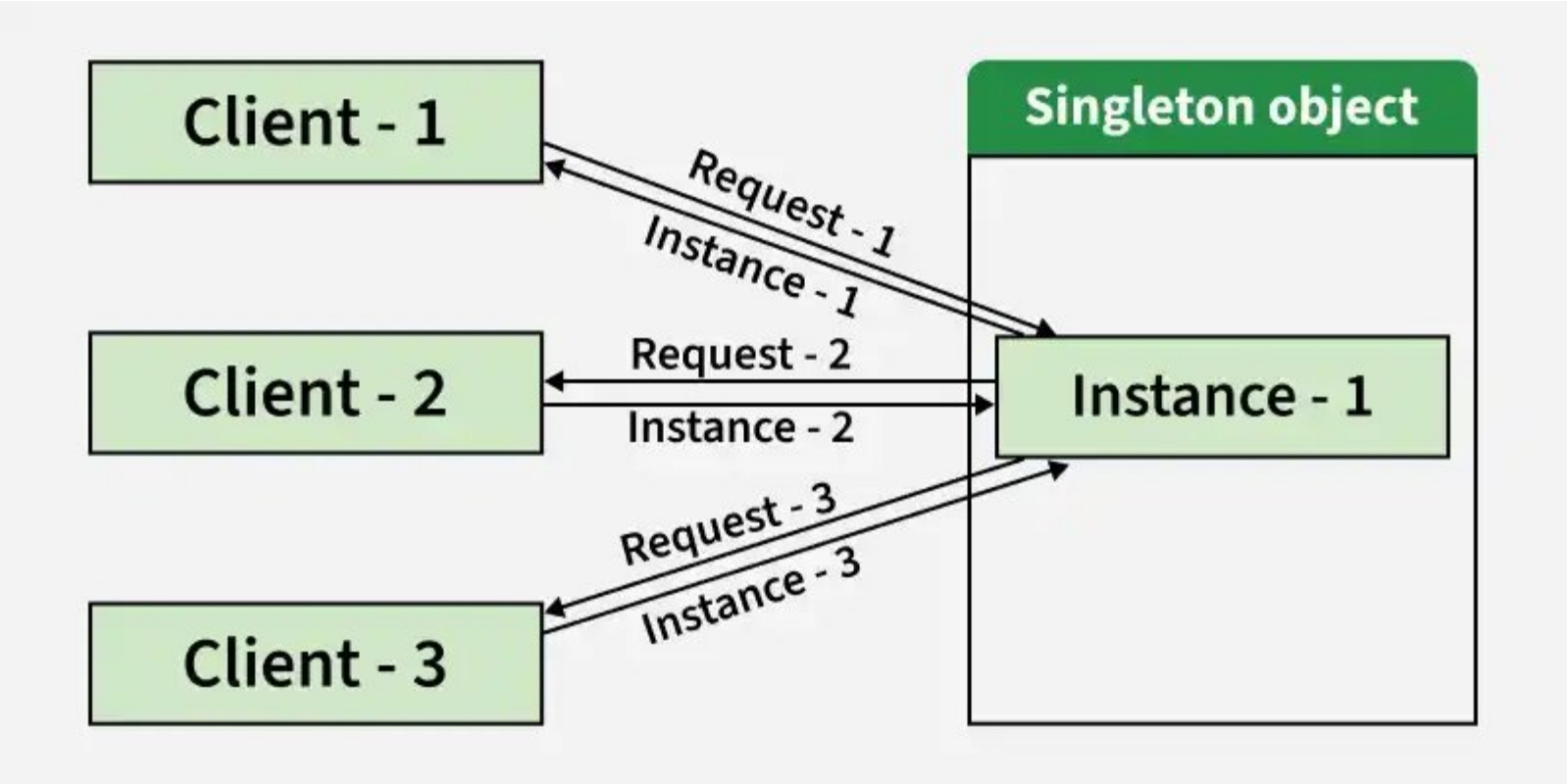
06 Flyweight Pattern

07 Proxy Pattern


Behavioral Design Patterns
01
Observer Pattern
02
Strategy Pattern
03
State Pattern
04
Command Pattern
05
Chain of Responsibility
06
Template Pattern
07
Interpreter Pattern
08
Visitor Pattern
09
Mediator Pattern
10
Memento Pattern

Application Of Design Patterns





Singleton is a creational design pattern that restricts a class to a single instance and provides a global access point to that instance.

Singleton Class in C++

```
#include <iostream>
using namespace std;

class Singleton {
private:
    Singleton() {
        cout << "Object created" << endl;
    }

public:
    void show() {
        cout << "Singleton object" << endl;
    }
};

int main() {
    // Singleton s;   //  Error: constructor is private

    return 0;
}
```

Real-Time Examples of Singleton Pattern

The **Singleton pattern** is useful when an application should have **only one shared instance** of a particular resource or manager.

```
class Logger {
public:
    void log() {
        cout << "Logging..." << endl;
    }
};

int main() {
    Logger l1;
    Logger l2;
}
```

l1 → Logger Object 1
l2 → Logger Object 2

```cpp
#include <iostream>
using namespace std;

class Logger {
private:
    Logger() {
        cout << "Logger object created\n";
    }

public:
    static Logger& getInstance() {
        static Logger obj;
        return obj;
    }

    void log() {
        cout << "Logging...\n";
    }
};
```

```cpp
int main() {

    Logger& l1 = Logger::getInstance();
    Logger& l2 = Logger::getInstance();

    l1.log();
    l2.log();

    cout << (&l1 == &l2);

    return 0;
}
```

```
Logger object created
Logging...
Logging...
1
```

Private constructor prevents direct object creation, while a static object inside `getInstance()` creates and maintains the single instance.

Real-life example of Singleton is a **Printer Manager in an office**.
Imagine an office has **one central printer**. Many employees can send print requests, but you don't want every employee to create a separate `PrinterManager` object that controls the same printer.

```
#include <iostream>
using namespace std;
class PrinterManager {
private:

    // Private constructor
    PrinterManager() {
        cout << "Printer Manager Created" << endl;
    }
public:
    // Provides access to the single object
    static PrinterManager& getInstance() {
        static PrinterManager manager;
        return manager;
    }
    void print(string document) {
        cout << "Printing: " << document << endl;
    }
};

int main() {

    PrinterManager& p1 = PrinterManager::getInstance();
    PrinterManager& p2 = PrinterManager::getInstance();

    p1.print("Assignment.pdf");
    p2.print("Report.pdf");

    return 0;
}
```

Printer Manager Created Printing: Assignment.pdf Printing: Report.pdf

Employee 1 → PrinterManager 1 → Printer
Employee 2 → PrinterManager 2 → Printer
Employee 3 → PrinterManager 3 → Printer



- `class Uncopyable` is designed as a root class that can make copy functions of any child class `private`

```
class Uncopyable {
    protected:                  // allow construction
        Uncopyable() { }  // and destruction of
        ~Uncopyable() { } // derived objects...
    private:
        Uncopyable(const Uncopyable&); // ...but prevent copying
        Uncopyable& operator=(const Uncopyable&);
};
```

- Any class that inherits class Uncopyable by `private`, will not have copy functionality:

```
class MyClass: private Uncopyable { // class no longer declares copy ctor or copy assign. operator
// ...
};

int main() {
    MyClass test1, test2;
    test2 = test1; // error: use of deleted function
                   // 'MyClass& MyClass::operator=(const MyClass&)'
}
```

Simple Composition

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};
class Car {
public:
     // Initializes this Car with 8 cylinders
    Car() : e_(8) { }

    // Start this Car by starting its Engine
    void start() { e_.start(); }
private:
    Engine e_;  // Car has-a Engine
};
int main() {
    Car c;

    c.start();
}
```

private Inheritance

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};
class Car : private Engine {  // Car has-a Engine
public:
     // Initializes this Car with 8 cylinders
    Car() : Engine(8) { }

    // Start this Car by starting its Engine
    using Engine::start;


};
int main() {
    Car c;

    c.start();
}
```

Constructors in Derived Classes

- We know that **constructor is invoked** implicitly when an **object is created**

- In **inheritance**, when we create object of derived class, what will happen?

Constructors in Derived Classes

Compiler's Default Constructor in case of inheritance

```
class A
{
public:
    /*A() //call after Derived class
    {
        //execute before Derived class
    }*/
};

class B: public A
{
public:
    /*B():A() //call before Base class
//it will call Base class's constructor first before executing body
    {
        //execute after Base class
    }*/
};

int main()
{
    B obj; //it will call child class(B)'s constructor
}
```

Constructors in Derived Classes

User's Default Constructor in case of inheritance

```
class A
{
public:
     A()
     {
          cout<<"Base class's Default constructor\n";
     }
};

class B: public A
{
public:
     B()    //B():A()
     {
          cout<<"Derived class's Default constructor";
     }
};

int main()
{
     B obj;
}
```

When the constructor of derived class `B` is called, explicitly call the default constructor `A()` of the base class `A` first.

```
Base class's Default constructor
Derived class's Default constructor
```

Constructors in Derived Classes

```
class A
{
    int a;
public:
    A(int k)
    {
        a=k;
    }
};

class B: public A
{
public:
    B()    //B():A()--> error: no matching function for call to A::A()
    {
        cout<<"Derived class's Default constructor";
    }
};

int main()
{
    B obj;
}
```

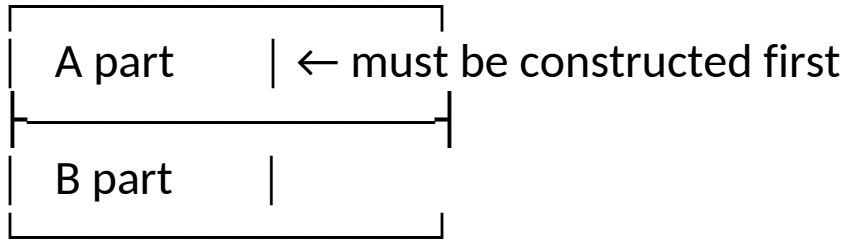
NOTE:

- As long as **no base class constructor takes any arguments**, the derived class need not have a constructor function.
- However, if any **base class contains a constructor with one or more arguments**, then it is mandatory for the derived class to **have a constructor, call base class's constructor explicitly and pass the arguments to the base class constructors**.

What happens internally?

To create `obj`, C++ must construct:

B object



C++ automatically tries to call the base constructor:

A(); But `A()` **does not exist**.

A(int k);

C++ cannot guess what value should be passed:

Should it be `A(10)`? `A(50)`? `A(0)`?

constructors in inheritance, especially what happens when the **base class has only a parameterized constructor**.

B has a default constructor, so B obj should work."

But **it will give an error**.

Why?
Whenever an object of a derived class is created, the **base-class constructor executes first**.
So:
`B obj;`

conceptually tries to do:

```
Create B object
      ↓
Call A constructor
      ↓
Call B constructor
```

Since we didn't explicitly specify which `A` constructor to call, C++ tries:
`A()`
But class `A` has:
`A(int k)`
and **does not have `A()`**.
Therefore, compilation error occurs.

Constructors in Derived Classes

```
class A
{
    int a;
public:
    A(int k)
    {
        a=k;
    }
};

class B: public A
{
public:
    B():A(5)
    {
    }
};

int main()
{
    B obj;
}
```

```
class A
{
    int a;
public:
    A(int k)
    {
        a=k;
    }
    void printA()
    {
        cout<<"a="<<a;
    }
};
```

```
a=2
b=3
```

```
class B: public A
{
    int b;
public:
    B(int x,int y):A(x)
    {
        b=y;
    }
    void printB()
    {
        cout<<"\nb="<<b;
    }
};

int main()
{
    B obj(2,3);
    obj.printA();
    obj.printB();
}
```

B obj
↓
A(5) ← Base constructor executes first
↓
B() ← Derived constructor executes second

A constructor
B constructor

```
class B : public A
{
public:
   B(int x) : A(x)
   {
      cout << "B constructor\n";
   }
};

int main()
{
   B obj(50);
}
```

```
B obj(50);
```

B(50)
↓
A(50)
↓
B constructor body

If the base class has only a parameterized constructor, the derived class must initialize the base class explicitly using the constructor initializer list**.**

`B(int x) : A(x)`

Think of it as:

Derived constructor → passes required data → Base constructor → Base object gets initialized → Derived constructor body executes.

Constructors in Derived Classes

```
class Parent1
{
    int a;
public:
    Parent1(int k)
    {
        a=k;
        cout<<"Parent1\n";
    }
};

class Parent2
{
    int b;
public:
    Parent2(int k)
    {
        b=k;
        cout<<"Parent2\n";
    }
};
```

```
class child: public Parent1, public Parent2
{
    int c;
public:
    child(int x,int y,int z):Parent2(x),Parent1(y)
    {
        c=z;
        cout<<"Child\n"<<endl;
    }
};

int main()
{
    child obj(2,3,4);
}
```

```
Parent1
Parent2
Child
```

NOTE: In case of **multiple inheritance**, the base class are constructed in the order in which they appear in the declaration of the derived class.

This is a **constructor initializer list for multiple inheritance**.

```
child(int x, int y, int z) : Parent2(x), Parent1(y)
{
   // child constructor body
}
```

```
Child obj(10, 20, 30);
```

Parent2(x) → Parent2(10)
Parent1(y) → Parent1(20)

Child(10,20,30)
  │
  ├── Parent2(10)
  ├── Parent1(20)
  └── Child body → z = 30

Very important: order is NOT determined by the initializer list
Even though you wrote:

: Parent2(x), Parent1(y)

the actual construction order is determined by the **inheritance declaration**:

class Child : public Parent1, public Parent2

Therefore, the actual order is:

```
Parent1(y)   ← constructed first
     ↓
Parent2(x)   ← constructed second
     ↓
Child        ← constructed last
```

Constructors in Derived Classes

```
class First
{
    int a;
public:
    First(int k)
    {
        a=k;
        cout<<"First\n";
        cout<<"a="<<a<<endl;
    }
};

class Second:public First
{
    int b;
public:
    Second(int x,int y):First(x)
    {
        b=y;
        cout<<"Second\n";
        cout<<"b="<<b<<endl;
    }
};
```

```
class Third: public Second
{
    int c;
public:
    Third(int x,int y,int z):Second(x,y)
    {
        c=z;
        cout<<"Third"<<endl;
        cout<<"c="<<c<<endl;
    }
};

int main()
{
    Third obj(2,3,4);
}
```

```
First
a=2
Second
b=3
Third
c=4
```

NOTE: In case of **multilevel inheritance**, the constructors will be executed in the order of inheritance.

Constructors in Derived Classes

- The **constructors for virtual base classes** are invoked before any non-virtual base classes.
- If there are **multiple virtual base classes**, they are invoked in the order in which they are declared. Any non-virtual bases are then constructed before the derived class constructor is executed.

Method of inheritance

```
class B:public A
{

};
class A:public B,public C
{

};
class A:public B,virtual public C
{

};
```

Order of execution

A(); base constructor
B(); derived constructor

B(); base(first)
C(); base(second)
A(); derived

C() ; virtual base
B(); ordinary base
A(); derived

Constructors in Derived Classes

- C++ supports **another method of initializing** the class objects.
- This method uses what is known as **initialization list** in the constructor function.

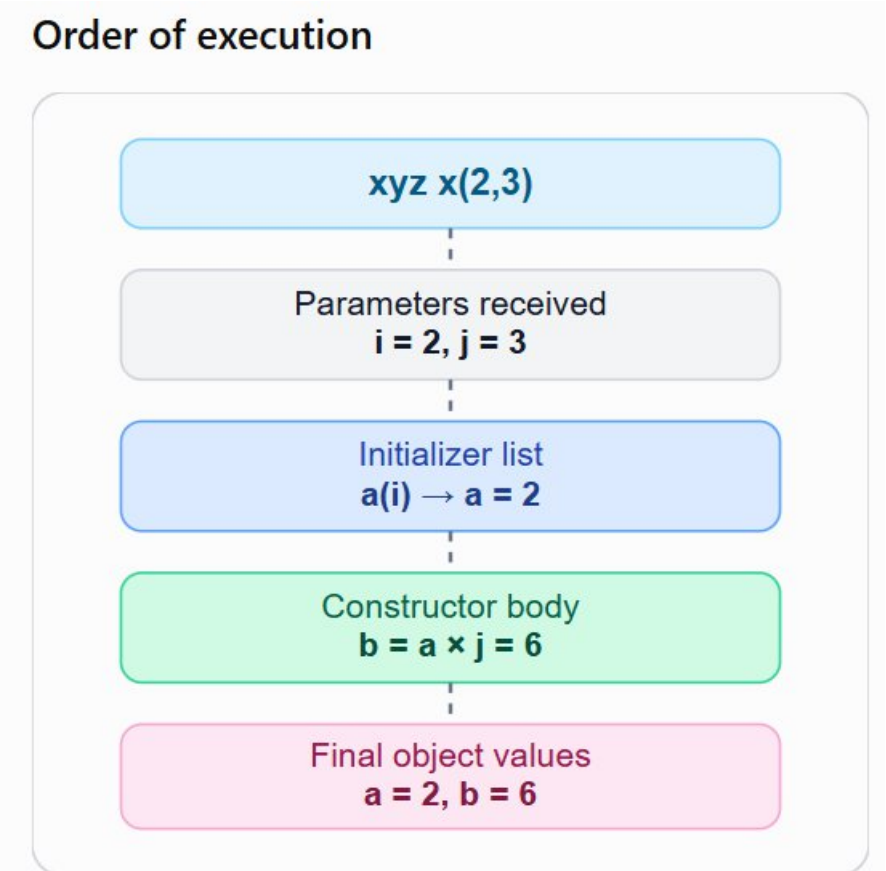
```
constructor(arglist):initialization-section
{
    assignment-section
}
```

- We can use **initialization-section** to provide **initial values to the base constructor and also to initialize its own class members.**
- We can use either/both section(assignment as well as initialization) to **initialize the data members of the constructor.**

Constructors in Derived Classes

```
class xyz
{
    int a,b;
public:
    xyz(int i,int j):a(i)
    {
        b=a*j;
    }
};

int main()
{
    xyz x(2,3);
    return 0;
}
```



Destructor in inheritance

```
class A
{
public:
     ~A()
     {
          cout<<"Base Class's Destructor";
     };
};

class B: public A
{
public:
    ~B()
     {
          cout<<"Derived Class's Destructor\n";
          //compiler will call Base class's Destructor
          //after executing Derived class Destructor's body
     };
};

int main()
{
     B obj; //it will call child class(B)'s destructor
}
```

```
Derived Class's Destructor
Base Class's Destructor
```

Practical : Multiple Inheritance

Create a **Class alpha** having data member: **int x** and **one argument constructor** which initializes the value of x. It also has member function which **displays** the value of x. Create another **class beta** which contains data member: **float y** and **one argument constructor** which initializes the value of y. It also has member function which **displays** the value of y. Create a **Class Gamma** which publicly inherits from class alpha and class beta and has two data members: **int m, n** and a **constructor** which passes argument to the base class constructor as well as initializes its own data members. Class Gamma also has member function to **print** the values of m and n. Write main function which creates object of class Gamma which passes values of base class constructor as well as derived class constructor.

Use the concept of **Multiple Inheritance** and **Constructor in Derived Class.**

Practical : Multiple Inheritance

```
class alpha
{
    int x;
public:
    alpha()
    {
        x=0;
        cout<<"\nDefault Constructor of alpha";
    }
    alpha(int n)
    {
        x=n;
        cout<<"\n\nParameterized Constructor of alpha";
    }
    void printx()
    {
        cout<<"\nValue of X: "<<x;
    }
};
```

```
class beta
{
    float y;
public:
    beta()
    {
        y=0;
        cout<<"\nDefault Constructor of beta";
    }
    beta(float n)
    {
        y=n;
        cout<<"\nParameterized Constructor of beta";
    }
    void printy()
    {
        cout<<"\nValue of Y: "<<y;
    }
};
```

Practical : Multiple Inheritance

```
class gamma:public alpha, public beta
{
    int m,n;
public:
    gamma()
    {
        cout<<"\nDefault Constructor of gamma";
        m=0;
        n=0;
    }
    gamma(int n1,float n2,int n3,int n4):beta(n2),alpha(n1)
    {
        m=n3;
        n=n4;
        cout<<"\nParameterized Constructor of gamma";
    }
    void printmn()
    {
        cout<<"\nValue of M and N is: "<<m<<" "<<n;
    }
};
```

```
int main()
{
    gamma g;
    g.printx();
    g.printy();
    g.printmn();

    gamma g1(1,2.5,3,4);
    g1.printx();
    g1.printy();
    g1.printmn();

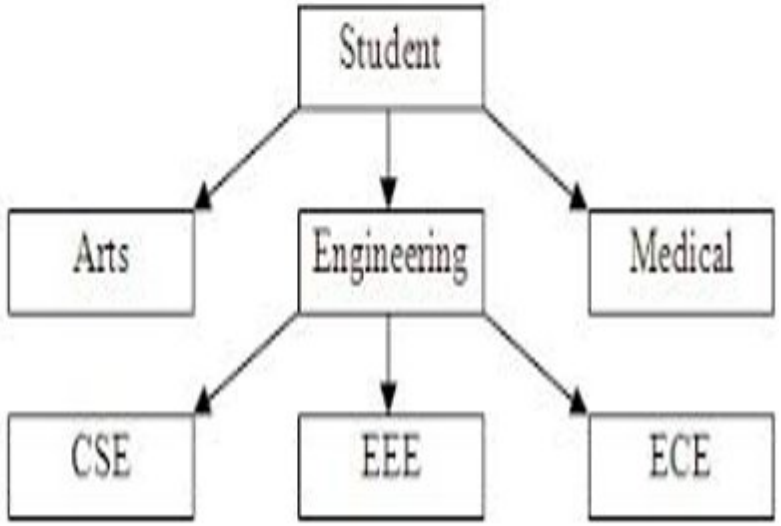
    return 0;
}
```

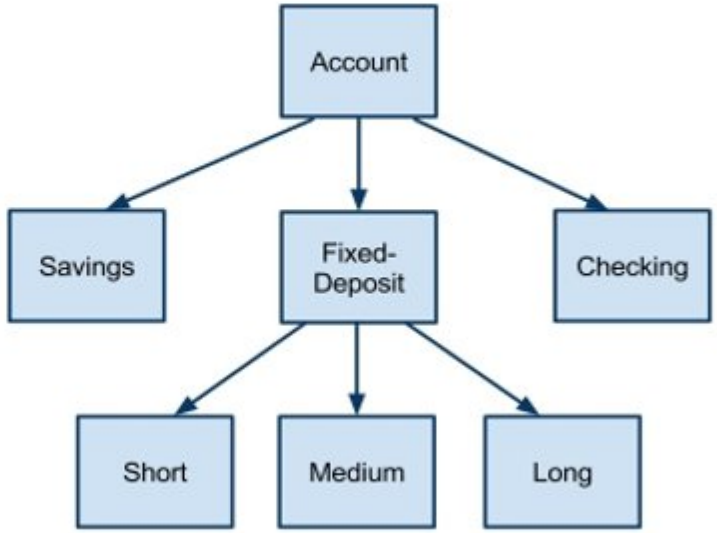
```
Default Constructor of alpha
Default Constructor of beta
Default Constructor of gamma
Value of X: 0
Value of Y: 0
Value of M and N is: 0 0

Parameterized Constructor of alpha
Parameterized Constructor of beta
Parameterized Constructor of gamma
Value of X: 1
Value of Y: 2.5
Value of M and N is: 3 4
```

Hierarchical Inheritance

- Many Programming problems can be cast into a **hierarchy** where certain **features of one level are shared by many others below that level**
- The **base class will include all the features that are common** to the subclasses.
- A **subclass can be constructed by inheriting the properties of the base class.**
- A **subclass can serve as a base class for the lower level classes** and so on





Hierarchical Inheritance

```
class Animal
{
public:
    void info()
    {
        cout << "I am an animal." << endl;
    }
};

class Dog : public Animal
{
public:
    void bark()
    {
        cout << "I am a Dog. Woof woof." << endl;
    }
};

class Cat : public Animal
{
public:
    void meow()
    {
        cout << "I am a Cat. Meow." << endl;
    }
};
```

```
int main()
{
    Dog dog1;
    cout << "Dog Class:" << endl;
    dog1.info();
    dog1.bark();

    Cat cat1;
    cout << "\nCat Class:" << endl;
    cat1.info();
    cat1.meow();
}
```

```
Dog Class:
I am an animal.
I am a Dog. Woof woof.

Cat Class:
I am an animal.
I am a Cat. Meow.
```

Practical :Hierarchical Inheritance

Write a program to create a **class Medicine** which stores **type of medicine, name of company, date of manufacturing**. **Class Tablet** is inherited from Medicine. Tablet class has **name of tablet, quantity per pack, price of one tablet** as members. **Class Syrup** is also inherited from Medicine and it has **quantity per bottle, dosage unit** as members. Both the classes contain necessary member functions for **input** and **output** data. Write a main( ) that enter data for tablet and syrup, also display the data.

Use the concepts of **Hierarchical Inheritance**.

Practical :Hierarchical Inheritance

```
class medicine
{
    char type_medicine[10],name_company[15],date_manufact[12];
public:
    void getdata()
    {
        cout<< "Enter the type of medicine (tablet/syrup) : ";
        cin>>type_medicine;
        cout<< "Enter the name of manufacturing company : ";
        cin>>name_company;
        cout<< "Enter the date of manufacturing (e.g. 21-Jan-2015) : ";
        cin>>date_manufact;
    }
    void putdata()
    {
        cout<<endl<< "Your medicine is " <<type_medicine;
        cout<<" manufactured by "<<name_company<<" on "<<date_manufact;
    }
};
```

Practical :Hierarchical Inheritance

```
class tablet : public medicine
{
    char name_tablet[30];
    int quantity_pack,price_per_tablet;
public:
    void gtdata()
    {
        cout<< "Enter name of tablet : ";
        cin>>name_tablet;
        cout<< "Enter number of tablets per pack : ";
        cin>>quantity_pack;
        cout<< "Enter the price of each tablet : ";
        cin>>price_per_tablet;
    }
    void ptdata()
    {
        cout<<endl<<name_tablet<< " has " <<quantity_pack;
        cout<<" tablets per pack having "<<price_per_tablet<<" Rs. price per tablet.";
    }
};
```

Practical :Hierarchical Inheritance

```
class syrup : public medicine
{
    int quantity_bottle,dosage_unit;
public:
    void gsdata()
    {
        cout<< "Enter the quantity of syrup per bottle in ml : ";
        cin>>quantity_bottle;
        cout<< "Enter the dosage unit : ";
        cin>>dosage_unit;
    }
    void psdata()
    {
        cout<<endl<< "One bottle of syrup is of " <<quantity_bottle;
        cout<<" ml and having " <<dosage_unit<< " dosage unit.";
    }
};
```

Practical :Hierarchical Inheritance

```
int main()
{
    tablet t;
    syrup s;

    t.getdata();
    t.gtdata();
    t.putdata();
    t.ptdata();
    cout<<endl<<endl;

    s.getdata();
    s.gsdata();
    s.putdata();
    s.psdata();

    return 0;
}
```

```
Enter the type of medicine (tablet/syrup) : tablet
Enter the name of manufacturing company : xyz
Enter the date of manufacturing (e.g. 21-Jan-2015) : 01-Jan-2021
Enter name of tablet : crocin
Enter number of tablets per pack : 6
Enter the price of each tablet : 5

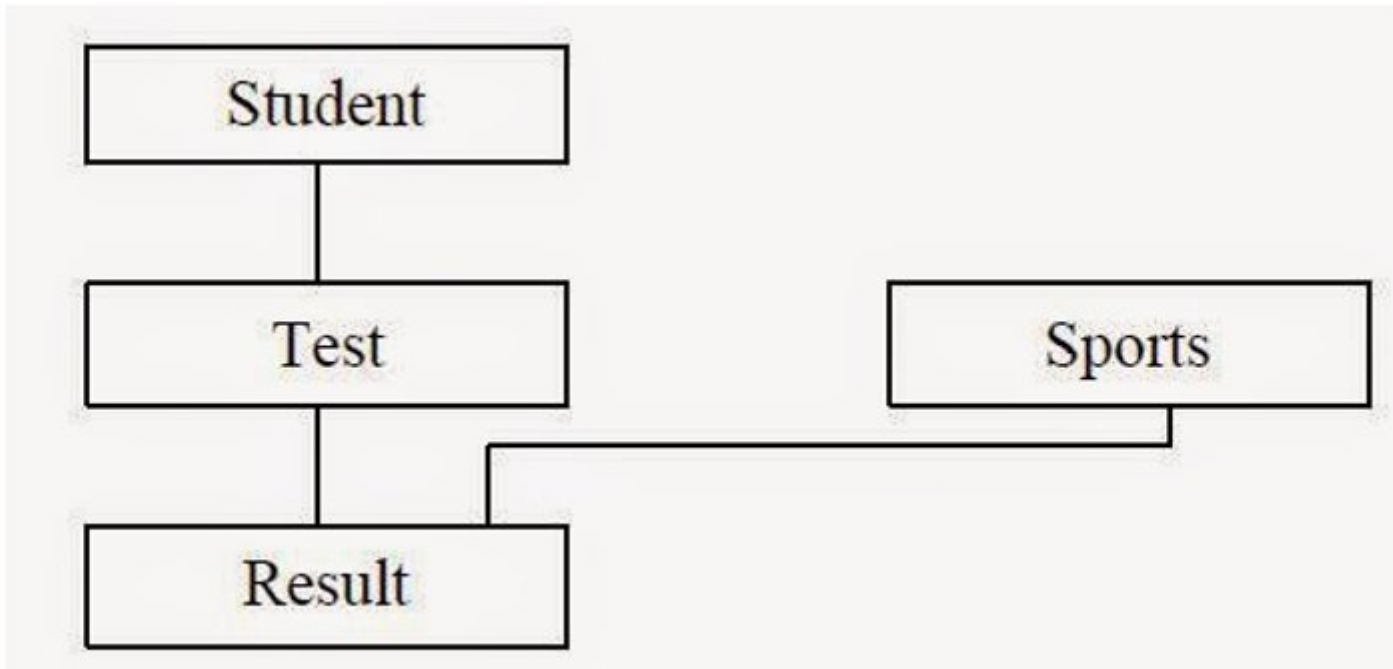
Your medicine is tablet manufactured by xyz on 01-Jan-2021
crocin has 6 tablets per pack having 5 Rs. price per tablet.

Enter the type of medicine (tablet/syrup) : syrup
Enter the name of manufacturing company : abc
Enter the date of manufacturing (e.g. 21-Jan-2015) : 07-Jan-2022
Enter the quantity of syrup per bottle in ml : 150
Enter the dosage unit : 20

Your medicine is syrup manufactured by abc on 07-Jan-2022
One bottle of syrup is of 150 ml and having 20 dosage unit.
```

Hybrid Inheritance

- There could be situations where we need to apply **two or more types of inheritance.**
- For instance, consider the following Fig. where result will have both **multilevel** and **multiple** inheritance



Hybrid Inheritance

```
class Student
{
    int rollno;
public:
    void getRoll(int a)
    {
        rollno=a;
    }
    void putRoll(void)
    {
        cout <<"Roll No: "<< rollno<<endl;
    }
};

class Test : public Student
{
protected:
    float subject1, subject2;
public:
    void getMarks(float x, float y)
    {
        subject1=x;  subject2=y;
    }
    void putMarks(void)
    {
        cout<< "\nMarks gain: "<<endl <<"Subject1 = ";
        cout<<subject1<<endl<<"Subject2 = "<<subject2 <<endl;
    }
};
```

Hybrid Inheritance

```
class Sport
{
protected:
     float score;
public:
     void getScore(float s)
     {
          score=s;
     }
     void putScore(void)
     {
          cout<<"Sports score: "<<score<<endl;
     }
};

class Result : public Test, public Sport
{
     float total;
public:
     void display(void);
};

void Result:: display(void)
{
     total=subject1+subject2+score;
     putRoll();
     putMarks();
     putScore();
     cout<<"\nTotal Score: "<<total<<endl;
}
```

```
int main()
{
     Result stu;
     stu.getRoll(10);
     stu.getMarks(40,50);
     stu.getScore(60);
     stu.display();
}
```

```
Roll No: 10

Marks gain:
Subject1 = 40
Subject2 = 50
Sports score: 60

Total Score: 150
```

Ambiguity Resolution in Inheritance

- **Ambiguity** may arise in inheritance application, when the **function in derived class overrides the inherited function.**
- A call to function with same name in base class and derived class by **derived class object** will **invoke function defined in derived class only**.
- However, we may invoke the function defined in base class by using the **scope resolution operator**.

Ambiguity Resolution in Inheritance

In Single Inheritance

```
class A
{
public:
     void display(void)
     {
          cout << "Class A \n";
     }
};

class B : public A
{
public:
     void display(void)
     {
          cout << "Class B \n";
     }
};

int main( )
{
    B b;                  // derived class object
    b.display( );         // invokes display() in B
    b.A::display( );      // invokes display() in A
    b.B::display( );      // invokes display() in B
}
```

```
Class B
Class A
Class B
```

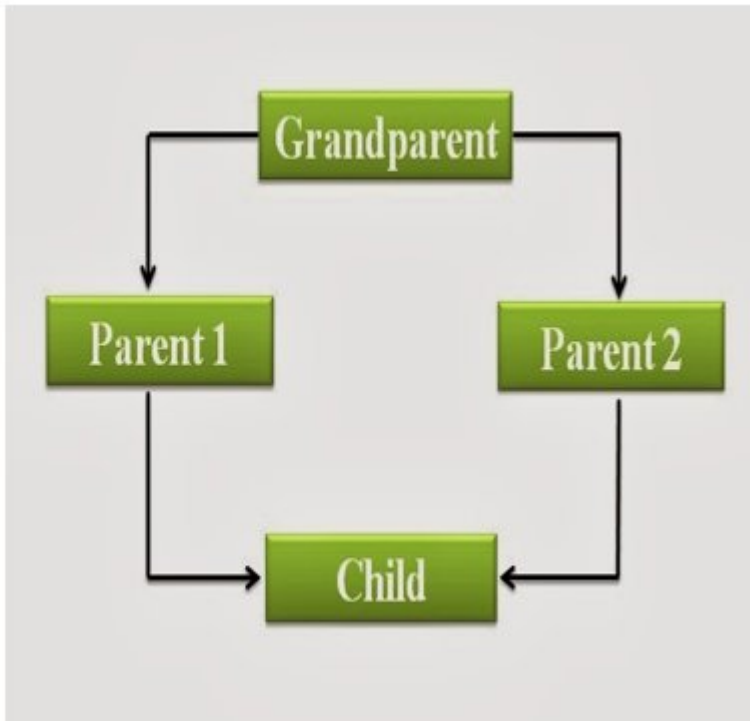
Ambiguity Resolution in Inheritance

- We may also face a problem in using the **multiple inheritance**, when a **function with the same name** appears in more than one base class.
- We can solve this problem by defining a named instance within derived class, using the **class resolution operator** with the function.

Virtual Base Classes

- Consider the situation where we have one class **Grandparent** .This class is inherited by two other classes **Parent1** and **Parent2**. Both these class are inherited into a new class **child** as shown in figure below.

- As we can see from the figure that data members/function of class Grandparent are inherited **twice** to class child. One through class Parent1 and second through class Parent1.



Virtual Base Classes

- When any data / function member of class Grandparent is accessed by an object of class child, **ambiguity arises** as to which data/function member would be called?

- One inherited through Parent1 or the other inherited through Parent2. This **confuses compiler and it displays error.**

Virtual Base Classes

```
class Grandparent
{
public:
    void show()
    {
        cout << "Hello\n";
    }
};

class Parent1 : public Grandparent
{
};

class Parent2 : public Grandparent
{
};

class child : public Parent1, public Parent2
{
};

int main()
{
    child object;
    object.show(); //error: request for member 'show' is ambiguous
}
```

Virtual Base Classes

- To resolve this ambiguity when class Grandparent is inherited in both class Parent1 and class Parent2, it is declared as **virtual base class** by placing a keyword **virtual** as :

```
Syntax 1:
class Parent1 : virtual public Grandparent
{
};

Syntax 2:
class Parent2 : public virtual Grandparent
{
};
```

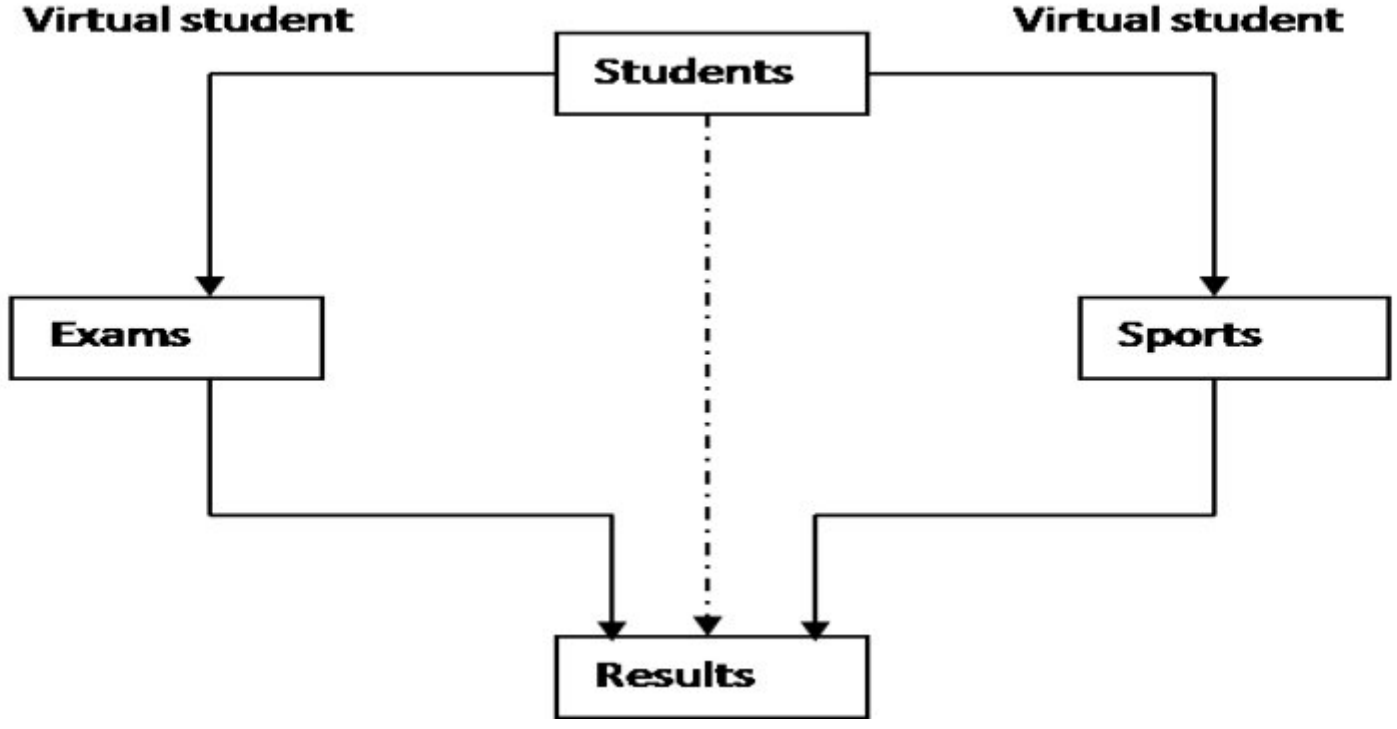
Note:

- virtual can be written **before or after the public**.
- Now **only one copy of data/function member will be copied** and class **Grandparent becomes the virtual base class.**

Virtual Base Classes

NOTE:

- When a class is made virtual base class, **C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance path exist between the virtual base class and a derived class**



Virtual Base Classes

```
class Student
{
     int rollno;
public:
     void getRoll(int a)
     {
          rollno=a;
     }
     void putRoll(void)
     {
          cout <<"Roll No: "<< rollno<<endl;
     }
};

class Test :virtual public Student
{
protected:
     float subject1, subject2;
public:
     void getMarks(float x, float y)
     {
          subject1=x;  subject2=y;
     }
     void putMarks(void)
     {
          cout<< "Marks gain: "<<endl <<"Subject1 ="<<subject1<<endl;
          cout<<"Subject2 = "<<subject2 <<endl;
     }
};
```

Virtual Base Classes

```
class Sport:public virtual Student
{
protected:
    float score;
public:
    void getScore(float s)
    {
        score=s;
    }
    void putScore(void)
    {
        cout<<"Sports score: "<<score<<endl;
    }
};

class Result : public Test, public Sport
{
    float total;
public:
    void display(void);
};

void Result:: display(void)
{
    total=subject1+subject2+score;
    putRoll();
    putMarks();
    putScore();
    cout<<"Total Score: "<<total<<endl;
}
```

```
int main()
{
    Result stu;
    stu.getRoll(10);
    stu.getMarks(40,50);
    stu.getScore(60);
    stu.display();
}
```

```
Roll No: 10
Marks gain:
Subject1 =40
Subject2 = 50
Sports score: 60
Total Score: 150
```

Virtual Function in C++

- A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.
- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call.
- They are mainly used to achieve Runtime polymorphism.
- Functions are declared with a **virtual** keyword in a base class.
- The resolving of a function call is done at runtime.

```
class Base{
public:
   virtual return_type  functionName(){
        // Function body
    }
}

class Derived : public Base{
public:
    // Overriden virtual
    // function of the base
    // class
    return_type functionName(){
       // Function body
    }
}
```

```
#include <iostream>
using namespace std;
class Base {
public:
    // Virtual function
    virtual void display() {
        cout << "Base class display" << endl;
    }
};
class Derived : public Base {
public:

    // Overriding the virtual
    // function in the derived class
    void display() {
        cout << "Derived class display" << endl;
    }
};
```

```
int main() {
    Base * basePtr;
    Derived derivedObj;

    // Base class pointer pointing
    // to derived class object

basePtr = &derivedObj;

    // Calling the virtual function
    basePtr->display();
    return 0;
}
```

```
Derived class display
```

Pure Virtual Function

- A virtual function is called a pure virtual function if it does not have **any implementation and is assigned = 0.** A class that contains a pure virtual function is called an abstract class.

```
#include <iostream>
using namespace std;
class Base {
public:
  // Pure virtual function
   virtual void display() = 0;
};

class Derived : public Base {
public:
   void display() override {
      cout << "Derived class display" << endl;
    }
};
```

```
int main() {
    Base* basePtr;
    Derived derivedObj;
    basePtr = &derivedObj;
    basePtr->display();
    return 0;
}
```

- Derived class display

Early Binding and Late Binding

- In layman's terms, when a function is called in the code, binding decides **which function gets executed based on the context such as the type of object or the function signature**. Binding happens at two levels:

- **Early Binding:** It happens when a function call is resolved during the program's compilation. This makes **it faster because** everything is decided early.
- **Late Binding:** It happens with virtual functions where the exact function to call is **decided at runtime**, depending on the actual object type. This is **slower because the program** has to figure it out while running.

```
#include <iostream>
using namespace std;
class base {
public:
   virtual void print() {
      cout << "print base "
      "class\n";
   }
   void show() {
      cout << "show base class\n";
   }
};
class derived : public base {
public:
  void print() {
   cout << "print derived class\n";
   }

void show() {
      cout << "show derived class\n";
   }
};
```

```
int main() {
   base* bptr;
   derived d;    bptr = &d;


   // Virtual function,  binded at runtime
   bptr->print();


   // Non-virtual function,  binded at compile time
   bptr->show();
   return 0;
}
```

```
print derived class
show base class
```

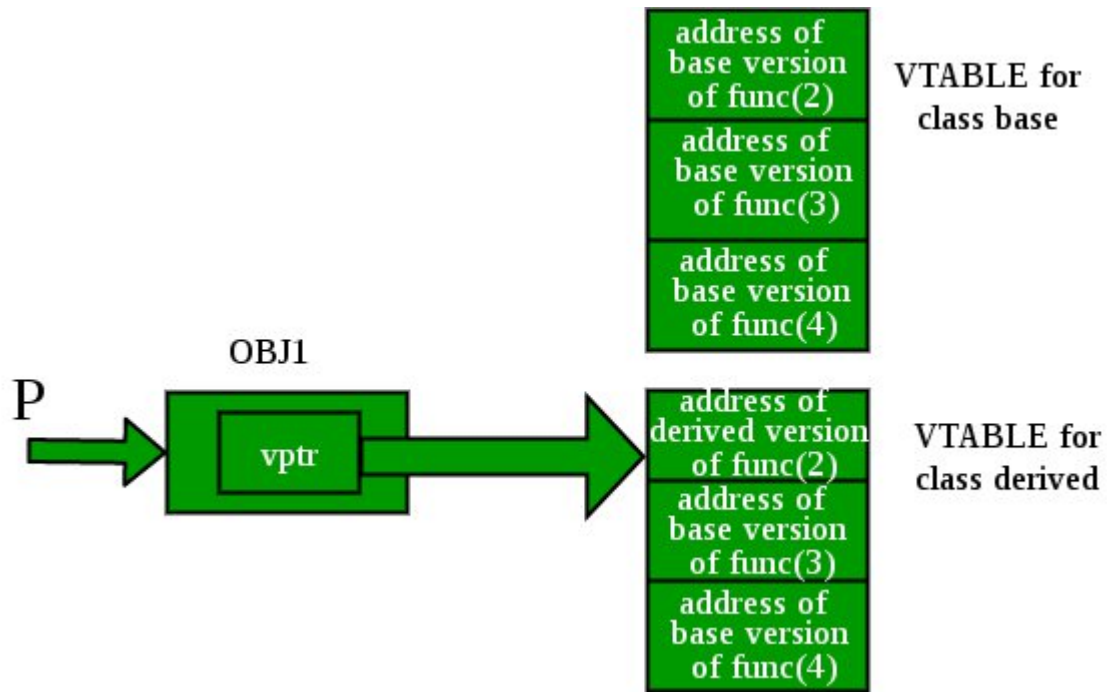
- In the above code, the print() function is declared with the virtual keyword so it will be bound at runtime and show() is non-virtual so it will be bound during compile time.

```
#include <iostream>
using namespace std;
class base {
public:
    void fun_1() { cout << "base-1\n"; }
    virtual void fun_2() { cout << "base-2\n"; }
    virtual void fun_3() { cout << "base-3\n"; }
    virtual void fun_4() { cout << "base-4\n"; }
};
class derived : public base {
public:
    void fun_1() { cout << "derived-1\n"; }
    void fun_2() { cout << "derived-2\n"; }
    void fun_4(int x) { cout << "derived-4\n"; }
};
int main() {
    base* p;
    derived obj1;
    p = &obj1;
    // Early binding because fun1() is non-virtual     // in base
    p->fun_1();
    // Late binding (RTP)
    p->fun_2();
    // Late binding (RTP)
    p->fun_3();
    // Late binding (RTP)
    p->fun_4();
    // Early binding but this function call is
    // illegal (produces error) because pointer
    // is of base type and function is of
    // derived class
    // p->fun_4(5);
    return 0;
}
```

```
base-1
derived-2
base-3
base-4
```

- Initially, we create a pointer of the type base class and initialize it with the address of the derived class object. When we create an object of the derived class, the compiler creates a pointer as a data member of the class containing the address of VTABLE of the derived class.

- A similar concept of Late and Early Binding is used as in the above example. For the fun_1() function call, the base class version of the function is called, fun_2() is overridden in the derived class so the derived class version is called, fun_3() is not overridden in the derived class and is a virtual function so the base class version is called, similarly fun_4() is not overridden so base class version is called.


address of base version of func(2)
address of base version of func(3)
address of base version of func(4)
VTABLE for class base
OBJ1
P
vptr
address of derived version of func(2)
address of base version of func(3)
address of base version of func(4)
VTABLE for class derived

Rules for Virtual Functions

- The rules for the virtual functions in C++ are as follows:
- Virtual functions cannot be static.
- A virtual function can be a friend function of another class.
- Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.
- The prototype of virtual functions should be the same in the base as well as the derived class.
- They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.
- A class may have a virtual destructor, but it cannot have a virtual constructor.

Limitations of Virtual Functions

- Slower: The function call takes slightly longer due to the virtual mechanism and makes it more difficult for the compiler to optimize because it does not know exactly which function is going to be called at compile time.
- Difficult to Debug: In a complex system, virtual functions can make it a little more difficult to figure out where a function is being called from.

- Design a base class shape with two double type values and member functions to input the data and compute_area() for calculating area of shape. Derive two classes: triangle and rectangle. Make compute_area() as an abstract function and redefine this function in the derived class to suit their requirements. Write a program that accepts dimensions of triangle/rectangle and displays calculated area. Implement dynamic binding for given case study.

```
#include <iostream>
using namespace std;

// Abstract base class
class Shape {
protected:
    double dim1, dim2; // common dimensions
public:
    // Function to input dimensions
    void getData(double a, double b) {
        dim1 = a;
        dim2 = b;
    }
 // Pure virtual function
    virtual double compute_area() = 0;
};

// Derived class for Rectangle
class Rectangle : public Shape {
public:
    double compute_area() override {
        return dim1 * dim2;  // length * breadth
    }
};

// Derived class for Triangle
class Triangle : public Shape {
public:
    double compute_area() override {
        return 0.5 * dim1 * dim2;  // 0.5 * base * height
    }
};
```

```
int main() {
    Shape *shape;   // base class pointer
    Triangle t;
    Rectangle r;

    int choice;
    double x, y;

    cout << "Choose Shape:\n1. Triangle\n2. Rectangle\nEnter choice:
";
    cin >> choice;

    cout << "Enter two dimensions: ";
    cin >> x >> y;

    if (choice == 1) {
      shape = &t;
    } else {
        shape = &r;
    }
  shape->getData(x, y); // set dimensions
    cout << "Area = " << shape->compute_area() << endl; // dynamic
binding

    return 0;
}
```

Choose Shape:
1. Triangle
2. Rectangle
Enter choice: 1
Enter two dimensions: 10 20
Area = 100

Choose Shape:
1. Triangle
2. Rectangle
Enter choice: 2
Enter two dimensions: 10 20
Area = 200

- shape = &t;
- shape is a pointer of type Shape* (base class pointer).
- t is an object of class Triangle (derived class).
- &t gives the address of the triangle object.
- So now, shape points to a Triangle object — even though its type is Shape*.

- shape->compute_area();
- Compiler sees compute_area() is virtual.
- At runtime, it looks at what shape is actually pointing to.
- Since shape points to a Triangle, the Triangle’s overridden function executes.

- shape = &r; // Now shape points to a Rectangle
- shape->compute_area(); // Rectangle's compute_area() runs

VIRTUAL FUNCTION

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects.
- But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.

- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

RULES FOR VIRTUAL FUNCTION

- Virtual functions **must be members of some class**.
- Virtual functions cannot **be static members.**
- They are accessed through **object pointers.**
- They can be a **friend of another class.**
- A virtual function must be defined in the base class, even though it is not used.
- The **prototypes of a virtual function of the base class and all the derived classes must be identical.** If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

Example

```
include <iostream>
{
 public:
 virtual void display()
 {
  cout << "Base class is invoked"<<endl;
 }
};
class B:public A
{
 public:
 void display()
 {
  cout << "Derived Class is invoked"<<endl;
 }
};
```

Derived class is derived from the base class.

```
int main()
{
 A* a;    //pointer of base class
 B b;     //object of derived class
 a = &b;
 a->display();   //Late Binding occurs
}
```

Output:

Derived Class is invoked

PURE VIRTUAL FUNCTION

- A virtual function **is not used for performing any task.** It only serves as a placeholder.
- When the function **has no definition, such function is known as "do-nothing" function.**
- The "do-nothing" function is known as **a pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.
- Pure virtual function can be defined as:

- virtual void display() = 0;

EXAMPLE

```
#include <iostream>
using namespace std;
class Base
{
   public:
   virtual void show() = 0;
};
class Derived : public Base
{
   public:
   void show()
   {
      std::cout << "Derived class is derived from the base class." << std::endl;
   }
```

```
int main()
{
    Base *bptr;
    //Base b;
    Derived d;
    bptr = &d;
    bptr->show();
    return 0;
}
```

Output:

Derived class is derived from the base class.

Practical : Hybrid Inheritance

Define a **class Hospital** having **rollno** and **name** as data members and member function to **get** and **print** data. Derive a **class Ward** from class Hospital having data members: **ward number** and member function to **get** and **print** data. Derive another **class Room** from Hospital having data member **bed number** and **nature of illness** and member function to **get** and **print** data. Derive **class Patient** from Class Ward and Class Room. In main () declare **5 object of Class Patient** and get and display all the information.

Use the concept of **Virtual Base Class** and **Hybrid Inheritance**.

Practical : Hybrid Inheritance

```
class hospital
{
    int rollno;
    char name[10];
public:
    void getdata()
    {
        cout<< "\nEnter the patient number : ";     cin>>rollno;
        cout<< "Enter the patient name : ";         cin>> name;
    }
    void putdata()
    {
        cout<< "patient number : " <<rollno<<endl;
        cout<< "patient name : " << name <<endl;
    }
};

class ward : virtual public hospital
{
    int ward_number;
public:
    void gwdata()
    {
        cout<< "Enter the ward number : ";        cin>>ward_number;
    }
    void pwdata()
    {
        cout<< "ward number : " <<ward_number<<endl;
    }
};
```

Practical : Hybrid Inheritance

```
class room : public virtual  hospital
{
    int bed_number;
    char illness[20];
public:
    void grdata()
    {
        cout<< "Enter the bed number : ";
        cin>>bed_number;
        cout<< "Enter the illness type : ";
        cin>> illness;
    }
    void prdata()
    {
        cout<< "bed number : " <<bed_number<<endl;
        cout<< "illness type : " << illness <<endl;
    }
};

class patient : public ward, public room
{

};
```

```
int main()
{
    patient p[5];
    int i;
    for(i=0;i<3;i++)
    {
        p[i].getdata();
        p[i].gwdata();
        p[i].grdata();
    }
    for(i=0;i<3;i++)
    {
        cout<<"\n" <<endl;
        p[i].putdata();
        p[i].pwdata();
        p[i].prdata();
    }
    return 0;
}
```

Practical : Hybrid Inheritance

```
Enter the patient number : 1
Enter the patient name : abc
Enter the ward number : 3
Enter the bed number : 4
Enter the illness type : flu

Enter the patient number : 2
Enter the patient name : xyz
Enter the ward number : 6
Enter the bed number : 1
Enter the illness type : cancer

Enter the patient number : 3
Enter the patient name : pqr
Enter the ward number : 5
Enter the bed number : 3
Enter the illness type : fracture


patient number : 1
patient name : abc
ward number : 3
bed number : 4
illness type : flu


patient number : 2
patient name : xyz
ward number : 6
bed number : 1
illness type : cancer


patient number : 3
patient name : pqr
ward number : 5
bed number : 3
illness type : fracture
```

Abstract Classes

- An **abstract class** is one that is **not used to create objects**.
- It is designed only to **act as a base class**(to be inherited by other classes)
- It is a design concept in program development and provides a base upon which other classes may built.

Member Classes: Nesting of Classes

- **Is-a relationship:** Inheritance is a Is-a relationship
  - **Ex:** BMW is-a car [Here, BMW & car are classes]

- **Has-a relationship:** Composition is a Has-a relationship
  - **Ex:** Car has-a engine [Here, Car & engine are classes]

- **What is Nested class?**
  - A nested class is a class that is **declared in another class**
  - It implements the concept of **"has-a"** relationship.

Member Classes: Nesting of Classes

Syntax 1

```
class enclosing
{
access-specifier:
    enclosing members;
    ..............
    class nested
    {
     access-specifier:
         nested members;
         ..............
    }obj;
};
```

Syntax 2

```
class alpha{...};
class beta{....};
class gamma
{
    alpha a; //a is object of alpha
    beta b;  //b is object of beta
    ......
};
```

Member Classes: Nesting of Classes

- **Defining Nested class inside the Enclosing class**
  - Nested class as private
  - Nested class as public

- **Defining Nested class outside the Enclosing class**

Defining Nested class inside the Enclosing class

Nested class as public

```
class outer
{
    int x;
public:
    class inner
    {
    private:
        int y;
    public:
        void get()
        {
            cout<<"Enter y:";  cin>>y;
        }
        void put()
        {
            cout<<"y="<<y<<endl;
        }
    };
    public:
        void geto()
        {
            cout<<"Enter x:";  cin>>x;
        }
        void puto()
        {
            cout<<"x="<<x;
        }
};
```

```
int main()
{
    //inner i;  --->ERROR
    outer::inner i;
    outer o;
    i.get();
    o.geto();
    i.put();
    o.puto();

    //o.i.get(); --->Error
    //o.i.put(); --->Error
    //To solve Error declare object i
    //in enclosing body of outer
}
```

```
Enter y:12
Enter x:32
y=12
x=32
```

Defining Nested class inside the Enclosing class

Nested class as public

```
class outer
{
     int x;
public:
     class inner
     {
     private:
          int y;
     public:
          void get()
          {
               cout<<"Enter y:";   cin>>y;
          }
          void put()
          {
               cout<<"y="<<y<<endl;
          }
     }i;
     public:
          void geto()
          {
               cout<<"Enter x:";   cin>>x;
          }
          void puto()
          {
               cout<<"x="<<x;
          }
};
```

```
int main()
{
     outer o;
     o.i.get();
     o.geto();
     o.i.put();
     o.puto();
}
```

```
Enter y:12
Enter x:32
y=12
x=32
```

Defining Nested class inside the Enclosing class

Nested class as private

```
class outer
{
     int x;
private:
     class inner
     {
     private:
          int y;
     public:
          void get()
          {
               cout<<"Enter y:";   cin>>y;
          }
          void put()
          {
               cout<<"y="<<y<<endl;
          }
     }i;
     public:
          void geto()
          {
               i.get();   //Solution of Error
               cout<<"Enter x:";   cin>>x;
          }
          void puto()
          {
               i.put();   //Solution of Error
               cout<<"x="<<x;
          }
};
```

```
int main()
{
     outer o;
     //o.i.get();   --->ERROR
     o.geto();
     //o.i.put();   --->ERROR
     o.puto();
}
```

```
Enter y:12
Enter x:32
y=12
x=32
```

Defining Nested class outside the Enclosing class

- C++ supports **another way of inheriting classes**:
  - **An object can be collection of many other objects** (A class can contain objects of other classes as its members)

```
class alpha{....};
class beta {....};
class gamma
{
    alpha a;
    beta b;
    .......
};
```

- All objects of gamma class will contain the objects a and b.
- This kind of relationship is called **containership** or **nesting**

Defining Nested class outside the Enclosing class

Creation of an object that contains another object is very different than the creation of an independent object.

- An independent object is created by its **constructor** when it is declared with arguments.
- **A nested object is created in two stages:**
  - The **member objects** are created using their respective constructors.
  - Then **ordinary members** are created.

Defining Nested class outside the Enclosing class

- This means Constructors of all the member objects should be called before its own constructor body is executed.

```
class gamma
{
    .......
    alpha a;
    beta b;
public:
    gamma(arglist):a(arglist1),b(arglist2)
    {
        //Assignment section(for ordinary other members)
    }
};
```

Defining Nested class outside the Enclosing class

```
class person
{
    char name[20];
    int age;
public:
    void get()
    {
        cout<<"Enter name:";   cin>>name
        cout<<"Enter age:";    cin>>age;
    }
    void put()
    {
        cout<<"name="<<name;
        cout<<"\nage="<<age;
    }
};
```

```
class employee
{
    person p;
    int sal;
public:
    void get()
    {
        p.get();
        cout<<"enter salary:";  cin>>sal;
    }
    void put()
    {
        p.put();
        cout<<"\nsalary="<<sal;
    }
};

int main()
{
    employee e;
    e.get();
    e.put();
}
```

```
Enter name:xyz
Enter age:10
enter salary:1000
name=xyz
age=10
salary=1000
```

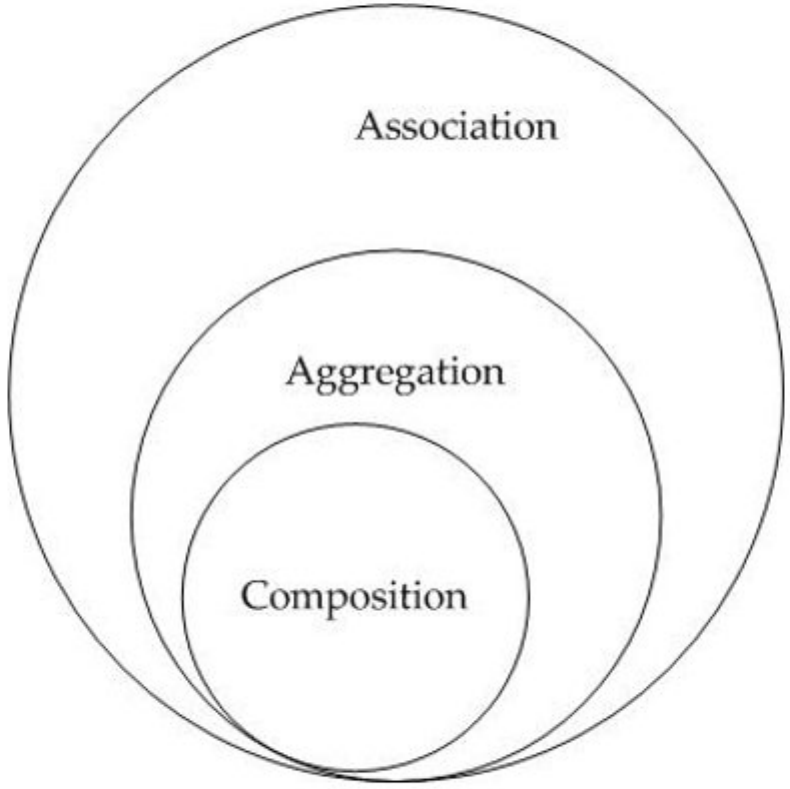
Association in C++

- **Association** is a relationship between two or more classes where objects of one class are related to objects of another class **without ownership**.
- Unlike **inheritance**, where one class derives from another, association defines **how objects interact** while keeping them independent.

Types of Association in C++

- **One-to-One Association**
  - A Person has a Passport
- **One-to-Many Association**
  - A Teacher teaches multiple Students
- **Many-to-One Association**
  - Many Employees work for One Company
- **Many-to-Many Association**
  - Students enroll in multiple Courses
- **Aggregation (Weak Association: "has-a")**
  - A Department has Professors
- **Composition (Strong Association: "part-of")**
  - A Car has an Engine

Relationship between Classes



Difference

Feature	Association	Aggregation	Composition	Inheritance
Definition	A general relationship between two independent classes.	A "**has-a**" relationship where one class contains another, but the contained object can exist independently.	A "**part-of**" relationship where one class owns another, and the contained object cannot exist without the owner.	A "**is-a**" relationship where one class derives from another, inheriting its properties and behaviors.
Dependency	No ownership (both objects exist independently).	Weak ownership (one object can exist without the other).	Strong ownership (if the container object is destroyed, contained objects are also destroyed).	Child class is fully dependent on the parent class.
Lifespan of Objects	Independent of each other.	Contained object can exist independently of the container.	Contained object **CANNOT** exist without the container.	Child class inherits properties and can extend or override behavior.
Example	A **Teacher** teaches multiple **Students**.	A **Department** has multiple **Professors**, but a **Professor** can exist even if the **Department** is deleted..	A **Car** has an **Engine**, and the **Engine** cannot exist without the **Car**.	A **Dog** is an **Animal** (Dog class inherits from Animal class).

4.11 Aggregation in C++

```cpp
#include <iostream>
#include <vector>
using namespace std;

class Professor
{
public:
    string name;
    Professor(string n) : name(n)
    {

    }
    ~Professor()
    {
        cout << "Professor " << name << " is destroyed!" << endl;
    }
    void show()
    {
        cout << "Professor " << name << " is still accessible!" << endl;
    }
};
```

```cpp
class Department
{
public:
    string deptName;
    vector<Professor*> professors; // Storing only pointers, not owning
    Department(string name) : deptName(name)
    {

    }
    // Adding Professor by reference
    void addProfessor(Professor &p)
    {
        professors.push_back(&p);
    }
    void showProfessors()
    {
        cout << deptName << " Department Professors: ";
        for (auto p : professors)
        {
            cout << p->name << " ";
        }
        cout << endl;
    }
    ~Department()
    {
        cout << deptName << " Department is destroyed!" << endl;
    }
};
```

4.11 Aggregation in C++

```cpp
#include <iostream>
#include <vector>
using namespace std;

class Professor
{
public:
    string name;
    Professor(string n) : name(n)
    {

    }
    ~Professor()
    {
        cout << "Professor " << name << " is destroyed!" << endl;
    }
    void show()
    {
        cout << "Professor " << name << " is still accessible!" << endl;
    }
};
```

```cpp
class Department
{
public:
    string deptName;
    vector<Professor*> professors; // Storing only pointers, not owning
    Department(string name) : deptName(name)
    {

    }
    // Adding Professor by reference
    void addProfessor(Professor &p)
    {
        professors.push_back(&p);
    }
    void showProfessors()
    {
        cout << deptName << " Department Professors: ";
        for (auto p : professors)
        {
            cout << p->name << " ";
        }
        cout << endl;
    }
    ~Department()
    {
        cout << deptName << " Department is destroyed!" << endl;
    }
};
```

4.11 Aggregation in C++

```
int main()
{
    Professor p1("Dr. John");
    Professor p2("Dr. Emma");

    {
        Department d1("Computer Science");
        d1.addProfessor(p1);
        d1.addProfessor(p2);


        d1.showProfessors();
    }  //Department goes out of scope and is destroyed

    cout << "Department is deleted but Professors are still accessible!" << endl;

    // Check if Professors still exist
    p1.show();
    p2.show();

    // Professors will be automatically destroyed at the end of `main()`
    return 0;
}
```

```
Computer Science Department Professors: Dr. John Dr. Emma
Computer Science Department is destroyed!
Department is deleted but Professors are still accessible!
Professor Dr. John is still accessible!
Professor Dr. Emma is still accessible!
Professor Dr. Emma is destroyed!
Professor Dr. John is destroyed!
```

4.9 Composition in C++

```cpp
#include <iostream>
using namespace std;

class Engine
{
public:
    Engine() { cout << "Engine Created" << endl; }
    ~Engine() { cout << "Engine Destroyed" << endl; }
};
class Car
{
private:
    Engine engine; // Composition: Car "has-a" Engine
public:
    Car() { cout << "Car Created" << endl; }
    ~Car() { cout << "Car Destroyed" << endl; }
};

int main()
{
    Car myCar;
    return 0;
}
```

```
Engine Created
Car Created
Car Destroyed
Engine Destroyed
```

Topics to be covered

4.1 Class and object

4.2 Set and Get Member Functions

4.3 Custom Constructors

4.4 Class Scope and Accessing Class Members

4.5 Access Functions and Utility Functions

4.6 Destructors

4.7 Default Assignment Operator

4.8 Const Objects and Const Member Functions

4.9 Composition: Objects as Members of Classes

4.10 Friend Classes

4.11 Aggregation

4.12 Base Classes and Derived Classes

4.13 Types of inheritance

4.14 Public, private and protected in Inheritance

4.15 Constructors and Destructors in Derived Classes

4.16 Relationship Between Base and Derived Classes

4.17 Constructors and Destructors in Derived Classes

Access Functions (Getters and Setters):

Purpose: To allow controlled access to a class's private data members.
Getters: Public member functions that return the value of a private member variable.
Setters: Public member functions that modify the value of a private member variable.

```
class MyClass
{
    private:
        int myData;
    public:
        int getMyData() { return myData; } // Getter
        void setMyData(int value) { myData = value; } // Setter
};
```

Utility Functions:

Purpose: To perform internal, non-public operations that support the public functionality of a class.
Accessibility: Typically declared as private member functions, making them inaccessible from outside the class.

```
class MyClass
{
    private:
        int myData;
        void internalOperation()
        { /* some internal logic */ } // Utility function
    public:
        void somePublicFunction()
        {
            internalOperation(); // Utility function called internally
        }
};
```

4.7 Default Assignment Operator

- The **assignment operator** (=) is used to **copy the values** from one object to another **existing** object of the same class.

Default Assignment Operator

- If you **don't define** your own assignment operator, **C++ provides a default one**. This operator performs a **member-wise (shallow) copy** of each data member from one object to another.

4.7 Default Assignment Operator

```cpp
#include <iostream>
using namespace std;

class Student
{
public:
    string name;
    int age;

    Student(){}
    Student(string n, int a) : name(n), age(a) {}
};

int main()
{
    Student s1("Alice", 20);
    Student s3;
    s3 = s1;          // Default assignment operator is used here

    cout << "s3 name: " << s3.name << ", age: " << s3.age << endl;
    return 0;
}
```

```
s3 name: Alice, age: 20
```

const objects

- We may create and use **constant objects** using **const** keyword before object declaration.

```
const matrix X(m,n);   //object X is constant
```

- Any **attempt to modify the values** of m and n will generate **compile-time error**.
- Further, a constant object can call only **const member functions.**
- As we know, a const member is a function prototype or function definition where the keyword **const** appears after the function's signature.
- Whenever const objects try to invoke **non-const member functions, the compiler generates errors**.

const objects

```cpp
class Student {
private:
    string name;
    int rollNumber;

public:
    // Constructor
    Student(string n, int r) : name(n), rollNumber(r) {}

    // Constant function: does NOT modify any data members
    void display() const {
        cout << "Name: " << name << ", Roll Number: " << rollNumber << endl;
    }

    // Non-constant function (cannot be called by a const object)
    void setRollNumber(int r) {
        rollNumber = r;
    }
};

int main() {
    const Student s1("Alice", 101); // Constant object

    s1.display();  // ✅ Allowed: Calls a const function

    // s1.setRollNumber(202); // ❌ Error: Cannot call non-const function on a const object

    return 0;
}
```

```
Name: Alice, Roll Number: 101
```

const objects

```cpp
class Student {
private:
    string name;
    int rollNumber;

public:
    // Constructor
    Student(string n, int r) : name(n), rollNumber(r) {}

    // Constant function: does NOT modify any data members
    void display() const {
        cout << "Name: " << name << ", Roll Number: " << rollNumber << endl;
    }

    // Non-constant function (cannot be called by a const object)
    void setRollNumber(int r) {
        rollNumber = r;
    }
};

int main() {
    const Student s1("Alice", 101); // Constant object

    s1.display();  // ✅ Allowed: Calls a const function

    // s1.setRollNumber(202); // ❌ Error: Cannot call non-const function on a const object

    return 0;
}
```

```
Name: Alice, Roll Number: 101
```

```
#include <iostream>
using namespace std;
// Base class
class Student
{
public:
    void display()
    {
        cout << "Student information" << endl;
    }
};
// First derived class
class Academic : public Student
{ public:
    void academicResult()
    {        cout << "Academic Result" << endl;
    }
};
// Second derived class
class Sports : public Student
{
public:
    void sportsResult()
    {
        cout << "Sports Result" << endl;
    }
};

// Derived class from Academic and Sports
class Result : public Academic, public Sports
{
public:
    void showResult()
    {
        cout << "Final Result" << endl;
    }
};

int main()
{
    Result r;

    r.academicResult();
    r.sportsResult();
    r.showResult();

    // Ambiguous statement
    // r.display();      // ERROR: ambiguous

    // Resolving ambiguity
    r.Academic::display();
    r.Sports::display();

    return 0;
}
```